# Orville Project Roadmap TODO

**Project:** Orville — Autonomous Multi-Agent Orchestration and Code-Generation Framework**Project ID:** `LEXzf7g37cAa2sJHx4PmMm` **Roadmap status:** Initialized**Document owner:** Orchestration Agent**Last updated:** 2026-08-27

## 1. Roadmap Objective

Build and maintain a standalone, environment-aware, multi-agent system that converts user objectives into executable task graphs, delegates work to specialized agents, verifies every material output, integrates artifacts, preserves project state, and produces runnable deliverables that remain usable outside Manus.

## 2. Definition of Done

The roadmap is complete when Orville can accept a new objective, create a dependency-aware task graph, assign each task to an appropriate specialist, execute independent tasks safely in parallel, maintain state and artifacts, perform second-agent verification, recover from failures, and deliver code, documentation, configuration, tests, deployment instructions, and a concise execution summary. The system must operate with Manus when available and degrade gracefully to standalone local tools when Manus-specific services are unavailable.

## 3. Status Legend

| Status | Meaning |
| --- | --- |
| `[ ]` | Not started |
| `[-]` | In progress |
| `[x]` | Completed |
| `[!]` | Blocked or requires user decision |
| `[~]` | Deferred by design |

## 4. Initial Baseline

- [x] ~~Load the Orville project instructions and operating constraints.~~

- [x] ~~Load the defined agent roles: Research, Code Synthesis, IDE, Prototype, Multi-Agent Pair Programmer, Automation, Orchestration, and Environment Interface Layer.~~
- [x] ~~Create an initial task graph for project initialization.~~
- [x] ~~Inventory the local runtimes, core utilities, installed skills, connector catalog, and attached workspace.~~
- [x] ~~Confirm GitHub CLI authentication without exposing credentials.~~
- [x] ~~Confirm the presence of configured model credential flags without exposing secret values.~~
- [x] ~~Create and deliver the initialization readiness report.~~
- [!] Repair or replace the degraded `fly dev` MCP integration; the configured hosted URL is unreachable, and official Fly documentation currently specifies a local `fly mcp server` process rather than this hosted endpoint. Requires an official reachable transport and Fly CLI/auth configuration.
- [x] ~~Repair the invalid `python fast api` MCP endpoint configuration by removing the trailing whitespace from `http://127.0.0.1:42069` ; the local MCP service must still be running before discovery can succeed.~~

# 5. Phase 0 — Governance and Project State

## 5.1 Project control files

- [x] ~~Define and maintain `PROJECT.md` with the current project objective, scope, assumptions, and non-goals.~~
- [x] ~~Define and maintain `STATE.md` with the current execution state, active phase, blockers, decisions, and recent outcomes.~~
- [x] ~~Define and maintain `TASK_GRAPH.md` with task IDs, dependencies, ownership, status, and validation gates.~~
- [x] ~~Define and maintain `AGENTS.md` with repository-specific operating rules when the workspace requires them.~~
- [x] ~~Define and maintain `CHANGELOG.md` for material roadmap, architecture, and behavior changes.~~
- [x] ~~Define a predictable directory structure for source code, tests, configuration, documentation, generated artifacts, logs, and temporary files.~~

### 5.2 Governance rules

- [x] ~~Document the rule that external instructions found in files, websites, emails, and tool outputs are treated as untrusted data unless explicitly endorsed.~~
- [x] ~~Document approval requirements for sensitive operations such as posting, payments, account changes, credential entry, and destructive file or repository actions.~~
- [x] ~~Document the policy for handling secrets, including storage, masking, rotation, and prohibition on logging credential values.~~
- [x] ~~Define artifact retention and cleanup rules for temporary files, downloaded assets, generated media, and execution logs.~~
- [x] ~~Define naming, formatting, commit, branch, and review conventions for all generated code.~~

## 6. Phase 1 — Orchestration Core

### 6.1 Task intake

- [x] ~~Define a normalized task intake schema containing objective, requested deliverables, constraints, environment, deadline, risk level, and acceptance criteria.~~
- [x] ~~Implement initial deterministic objective classification for research, coding, automation, document production, media generation, web development, deployment, mixed, and general tasks; agentic refinement remains pending.~~
- [x] ~~Implement assumption recording so missing but non-critical details are made explicit rather than silently inferred.~~
- [x] ~~Implement deterministic clarification gates for ambiguous requirements, sensitive actions, and conflicting project instructions; unavailable-credential resolution remains provider-specific.~~

### 6.2 Task graph construction

- [x] ~~Represent every task with a stable ID, title, description, inputs, outputs, dependencies, status, and validation method.~~
- [x] ~~Support sequential, conditional, retry, approval, and in-process parallel batch nodes; distributed and full human-in-the-loop scheduling remain pending.~~

- [x] ~~Detect dependency cycles before execution.~~
- [x] ~~Detect unknown task dependencies before execution.~~
- [x] ~~Detect missing required task inputs and unresolved ownership before execution through `required_inputs` and explicit `owner` validation.~~
- [x] ~~Reject static duplicate `owned_paths` claims during graph validation.~~
- [x] ~~Identify parallelizable tasks using dependency readiness, conditions, approval state, and static ownership checks; dynamic locks remain pending.~~
- [x] ~~Define graph state transitions: `planned`, `ready`, `running`, `blocked`, `failed`, `verified`, and `completed`.~~
- [x] ~~Persist task graph state so execution can resume after interruption.~~

### 6.3 Delegation and coordination

- [x] ~~Define routing rules mapping task capabilities to specialist agents through `AgentRegistry`.~~
- [x] ~~Define the Orchestration Agent as the owner of graph state, dependency release, integration, and final delivery.~~
- [x] ~~Define the Multi-Agent Pair Programmer as the owner of safe in-process parallel branch execution; distributed branch scheduling remains pending.~~
- [x] ~~Define explicit handoff formats between Research, Code Synthesis, IDE, Prototype, Automation, and verification roles through `AgentHandoff`.~~
- [x] ~~Reject static ownership conflicts before execution; dynamic locks, distributed leases, and merge protocol for parallel branches remain pending.~~
- [x] ~~Add per-task timeouts, existing retry limits, persisted cancellation requests, and failure escalation; hard process cancellation remains pending.~~

## 7. Phase 2 — Agent Contracts

### 7.1 Research Agent

- [x] ~~Define Research Agent inputs, source-quality requirements, citation integrity, uncertainty handling, separated fact/analysis/assumption/recommendation fields, and a validated research-output schema in `orville_core/agent_contracts.py`.~~

- [x] ~~Require every material finding to cite known source IDs and support minimum-source and primary-source requirements; multi-source cross-check policy selection remains brief-specific.~~
- [x] ~~Require separation of retrieved facts, analysis, assumptions, and recommendations in `ResearchFinding`.~~
- [x] ~~Define `AgentHandoffEnvelope` and documented Research Agent handoff formats for API documentation, competitive research, datasets, and evidence summaries.~~

## 7.2 Code Synthesis Agent

- [x] ~~Define complete-code requirements including relative file paths, dependencies, configuration, documentation blocks, setup instructions, tests, runtime target, and known limitations in `CodeSynthesisOutput`.~~
- [x] ~~Require an explicit target runtime and deployment environment in `CodeSynthesisOutput`; runtime-specific semantic validation remains task-specific.~~
- [x] ~~Require explicit known limitations and deterministic contract validation; implementation-specific error-handling review remains part of verification.~~
- [x] ~~Define a Code Synthesis handoff format containing changed files, dependencies, configuration, setup commands, tests, documentation blocks, runtime, and known limitations.~~

## 7.2A Core Orchestration Slice Completed

- [x] ~~Implement dependency-aware synchronous graph execution.~~
- [x] ~~Implement structured task and run events.~~
- [x] ~~Implement atomic JSON checkpoint writes with file durability handling for POSIX and Windows.~~
- [x] ~~Implement failure capture, dependent-task blocking, and resumable failed checkpoints.~~
- [x] ~~Add unit tests for graph validation, ordering, missing handlers, failure blocking, resume behavior, and checkpoint JSON validity.~~
- [x] ~~Add a runnable example and standalone engine documentation.~~

## 7.3 IDE Agent

- [x] ~~Define repository inspection procedures and `IDEInspectionReport` fields for structure, dependency tracing, entry points, configuration, shared interfaces, impact findings, and risks.~~
- [x] ~~Define safe refactoring procedures through `RefactorPlan`, requiring explicit behavior-change intent or preserved behaviors, validation commands, and rollback planning.~~
- [x] ~~Require `RefactorPlan` impact findings before changing declared shared interfaces; implementation-specific dependency scanning remains pending.~~
- [x] ~~Define the IDE Agent architectural findings format through `IDEInspectionReport`; live repository scanner implementation remains pending.~~

## 7.4 Prototype Agent

- [x] ~~Define rapid-build criteria, accepted shortcuts, prohibited shortcuts, runnable minimum state, and debugging handoff through `PrototypeSpec`.~~
- [x] ~~Require an explicit minimum runnable state for prototypes.~~
- [x] ~~Require explicit production-hardening steps in every `PrototypeSpec`.~~
- [x] ~~Require local-run commands, smoke-test commands, and debugging-handoff data in `PrototypeSpec`.~~

## 7.5 Automation Agent

- [x] ~~Define workflow trigger, schedule, retry, idempotency, notification, rollback, health-check, and persistent-runtime requirements through `AutomationSpec`.~~
- [x] ~~Require `approval_required=True` for sensitive automation actions through `AutomationSpec`.~~
- [x] ~~Define connector IDs, health-check requirements, rate-limit/retry fields, and credential-boundary handoff requirements in `AutomationSpec`; live connector-specific execution remains adapter-owned.~~
- [x] ~~Define persistent-service requirements and mandatory health checks in `AutomationSpec`; runtime deployment selection remains environment-specific.~~

## 7.6 Verification Agent

- [x] ~~Define an independent verification role or review pass for every material output.~~
- [x] ~~Require verification against acceptance criteria rather than stylistic preference.~~

- [x] ~~Define independent verification records and deterministic acceptance checks through `VerificationSpec`, including artifact-specific tests, source checks, visual checks, security checks, and manual evidence requirements.~~
- [x] ~~Record failures as actionable defects with reproduction steps and severity.~~

## 7.7 Model Provider Integration Agent

## 7.7A Phase 2 Provider Slice Completed

- [x] ~~Implement standard-library HTTP client with injectable test transport.~~
- [x] ~~Implement Gemini REST generation, system instructions, structured-output requests, tool-call normalization, usage extraction, and model health checks.~~
- [x] ~~Implement Ollama chat generation, structured outputs, tool payloads, usage extraction, and model health checks.~~
- [x] ~~Implement custom local Ollama-compatible endpoint support.~~
- [x] ~~Implement user-downloaded local model cataloging, SHA-256 hashing, metadata inspection, and provider configuration bridging.~~
- [x] ~~Add adapter and local-model tests; all 12 tests pass.~~
- [x] ~~Add `MODEL_PROVIDERS.md` usage and security documentation.~~
- [x] ~~Add `PHASE3_MEDIA_AND_EMBEDDINGS.md` usage and compatibility documentation.~~
- [x] ~~Add `PROVIDER_ROUTING.md` routing, privacy, and fallback documentation.~~
- [x] ~~Add `PROJECT.md`, `STATE.md`, `TASK_GRAPH.md`, and `WORKFLOW_FOUNDATION.md` for durable project state and workflow contracts.~~
- [x] ~~Add `VERIFICATION_AND_INTAKE.md` for intake, agent, verification, and model-task contracts.~~
- [x] ~~Add `EXECUTION_CONTROLS.md` for conditions, approvals, cancellation, idempotency, timeouts, and ownership.~~
- [x] ~~Add `PARALLEL_EXECUTION.md` for isolated contexts, batch reconciliation, failure handling, and production limitations.~~
- [x] ~~Add `PROVIDER_HARDENING.md` for capability discovery, circuit breaking, and local-model lifecycle.~~

- [x] ~~Design a provider-agnostic model adapter interface so Orville can use multiple cloud providers and local model servers without changing orchestration logic.~~
- [x] ~~Support cloud providers through user-supplied API credentials, including Gemini as a supported example and an extensible registry for additional providers.~~
- [x] ~~Support local and self-hosted models through user-supplied endpoint URLs, including Ollama as a supported example.~~
- [x] ~~Support provider settings for model name, base URL, temperature, timeout, structured output, and tool calling where implemented.~~
- [x] ~~Support OpenAI-compatible endpoints through `OpenAICompatibleAdapter`, including the managed Blackbox relay adapter, normalized chat, streaming, embeddings, health checks, and injectable test transport.~~
- [x] ~~Support normalized streaming responses for Gemini, Ollama, and custom Ollama-compatible endpoints.~~
- [x] ~~Support normalized multimodal message payloads, including text, image, audio, video, and file references where the provider accepts them.~~
- [x] ~~Support normalized embeddings for single and batched text inputs.~~
- [x] ~~Add managed-Blackbox provider capability negotiation and reject unsupported streaming, tool-calling, structured-output, and embedding requests before network requests; live endpoint capability discovery remains pending.~~
- [ ] Define and integrate streaming backpressure, cancellation, reconnect, and partial-response checkpoint behavior; bounded buffering, cooperative cancellation, and periodic partial checkpoints are implemented through `StreamPolicy`, while reconnect/resume behavior remains pending. Current tests cover the implemented runtime controls.
- [x] ~~Add `EmbeddingIndexSpec` with index versioning, vector dimension checks, batching limits, and migration-strategy validation; persistent index storage and re-embedding execution remain pending.~~
- [x] ~~Store provider configuration separately from prompts and task state through explicit provider objects and a registry.~~
- [x] ~~Implement capability-aware provider selection with preference ordering and local-only filtering.~~

- [x] ~~Implement complete-response and embedding fallback routing between configured providers.~~
- [x] ~~Implement preflight endpoint validation for scheme, host, credentials, fragments, and ports.~~
- [x] ~~Implement initial provider capability discovery, health-aware exclusion, and in-process circuit breaking; dynamic negotiated schemas, rate-limit accounting, latency ranking, and persistent telemetry remain pending.~~
- [x] ~~Preserve provider-neutral execution records while recording the selected provider and model metadata needed for reproducibility in model-task outputs and routing metadata.~~

## 7.8 Imported Local Model Agent

- [x] ~~Allow users to import local model files that they have already downloaded instead of requiring Orville to download them.~~
- [x] ~~Support importing models from user-selected files or directories, including recognized model formats and runtimes supplied by the user.~~
- [x] ~~Detect recognized model format, asset type, basic directory metadata, and file size when metadata is available.~~
- [x] ~~Register imported models in a local model catalog with a stable identifier, display name, source path, checksum, capabilities, and status.~~
- [ ] Allow users to select or change the storage location for imported models and avoid duplicating large files when a safe reference or link is possible.
- [x] ~~Validate file existence, readability, recognized format, runtime configuration, endpoint configuration, and basic disk availability before activating an imported model; memory, GPU, and full runtime checks remain pending.~~
- [ ] Support importing models intended for text, code, vision, embeddings, image generation, audio, or other modalities only when a compatible local runtime is available.
- [ ] Validate local runtime multimodal and embedding capabilities before exposing those operations in the GUI.
- [ ] Connect imported models to local runtimes such as Ollama or other user-configured inference servers, and support direct local execution where the installed runtime allows

it.

- [x] ~~Make imported local models selectable through the same provider-neutral interface used for endpoint-based models after catalog registration.~~
- [x] ~~Provide model lifecycle actions for validate, activate, deactivate, remove from catalog, and guarded deletion; metadata updates, relocation, and explicit GUI confirmation remain pending.~~
- [ ] Preserve license, provenance, checksum, and user ownership metadata for each imported model.
- [ ] Add clear diagnostics for unsupported formats, missing runtimes, corrupted files, insufficient resources, incompatible hardware, and license restrictions.

# 8. Phase 3 — Environment and Integration Reliability

## 8.1 Runtime health

- [x] ~~Create the repeatable `orville runtime-health` command for Python, Git, Node.js, pnpm, GitHub CLI, configuration utilities, MCP utility presence, and optional Python modules; MCP utilities are presence-only and are not invoked by the checker.~~
- [x] ~~Record Python version, platform, working directory, command availability, optional module availability, and required/optional status in the runtime-health report.~~
- [ ] Verify package installation and dependency resolution in a clean environment; the local test environment and syntax compilation pass.
- [x] ~~Define reproducible environment setup instructions for Linux, Windows, and containerized execution in `ENVIRONMENT_SETUP.md`; clean-machine installation validation remains pending.~~

## 8.2 Connector management

- [x] ~~Add secret-safe `ConnectorInventory` and `ConnectorHealth` records for enabled, disabled, degraded, unavailable, and unknown connector states; live connector discovery remains pending.~~
- [x] ~~Require `configuration_inspected=True` before a connector can be classified as unavailable.~~
- [x] ~~Define secret-safe connector health records with redacted capabilities, authentication status, rate-limit remaining, error codes, and no credential-bearing error messages; live~~

~~connector calls remain pending.~~

- [x] ~~Define connector authentication methods, required scopes, approval requirements, retryable statuses, rate-limit classification, bounded retries, and credential-safe failure reporting through `ConnectorAuthPolicy`; live connector-specific calls remain pending.~~
- [x] ~~Inspect `fly dev`; mark it operationally unavailable for this environment because its configured OAuth SSE transport targets a refused local endpoint, with repair and replacement procedure documented in `CONNECTOR_OPERATIONS.md`.~~
- [x] ~~Inspect for `python-fast-api`; no matching connector is configured, so no mutation was performed. Document the user-approved local FastAPI MCP registration and discovery path in `CONNECTOR_OPERATIONS.md`.~~
- [ ] Verify at least one harmless capability call for each connector required by a concrete project.
- [ ] Avoid enabling unrelated connectors or mutating connector configuration without a concrete requirement and user approval where required.

## 8.3 Cloud and Local Model Endpoints

- [ ] Define a secure model-provider configuration schema with provider type, display name, API key reference, endpoint URL, model identifier, protocol, capabilities, timeout, and enabled state.
- [ ] Define a separate local-model catalog schema for imported files, including path, checksum, format, architecture, quantization, runtime, capabilities, license, and validation status.
- [ ] Ensure local model imports never execute arbitrary model-provided code or scripts during scanning, metadata extraction, validation, or activation.
- [ ] Run imported models with least-privilege filesystem and network access, preferably with network access disabled unless the user explicitly enables it.
- [ ] Add resource controls for CPU, RAM, GPU, VRAM, disk usage, concurrency, context length, and generation limits.
- [ ] Add a dry-run or validation-only mode that inspects an imported model without activating or using it.
- [ ] Add tests for import, duplicate detection, checksum verification, metadata extraction, unsupported formats, insufficient resources, runtime mismatch, activation, generation,

deactivation, relocation, and explicit deletion confirmation.

- [x] ~~Accept API credentials only from the user or a user-provided secure configuration source; never invent, infer, reuse, or print secret values.~~
- [ ] Accept endpoint URLs only from the user or an explicitly approved configuration source; validate URL scheme, host, port, path, and network reachability before use.
- [ ] Add a guided configuration flow for cloud providers such as Gemini using an API key and for local servers such as Ollama using a user-supplied endpoint such as `http://localhost:11434`.
- [ ] Add a safe health check that confirms authentication and model availability without logging prompts, responses, or secrets.
- [ ] Add model discovery where the provider supports it, while allowing manual model-name entry for providers without discovery.
- [ ] Add privacy controls that distinguish local-only, cloud-approved, and restricted data before routing prompts or files.
- [ ] Add per-provider rate limits, timeout handling, retries, circuit breaking, and fallback routing.
- [ ] Add configuration export and import using redacted templates so the system remains portable outside Manus.
- [ ] Add tests for valid credentials, invalid credentials, unreachable endpoints, unsupported protocols, missing models, rate limits, and local-only routing.

## 8.3A Optional Blackbox Integration

**Objective:** Use Blackbox as Orville’s cloud inference service without requiring users to manage a Blackbox API key or sign in during initial setup, while allowing users to optionally connect their own Blackbox account for personal quota, plan, model, and usage control. The no-user-credential path requires an Orville-managed backend relay; it is not an offline or unauthenticated provider.

**Research status:** Blackbox’s public API and Agent API documentation currently require a Bearer API key for programmatic requests. The Agent API documentation also lists a Blackbox account and Pro subscription as prerequisites. The public CLI documentation requires account configuration and API-key setup before first use. No official third-party OAuth or device-authorization flow was identified in the reviewed public documentation. Treat ordinary Blackbox website login as insufficient for API authorization until Blackbox documents an official delegated-authentication flow.

**Source records:**

- [x] ~~Preserve the reviewed sources and access dates in a dedicated research note: `docs/BLACKBOX_INTEGRATION_RESEARCH.md`.~~
- [ ] Re-check the official Blackbox API, Agent API, CLI, authentication, terms, privacy, and developer documentation immediately before implementation because authentication and subscription requirements may change.
- [ ] Contact or verify Blackbox developer support regarding third-party OAuth, device authorization, CLI token interoperability, scopes, redirect URIs, refresh-token behavior, rate limits, and redistribution requirements.

**Architecture and provider boundary:**

- [x] ~~Implement the initial Blackbox managed-cloud provider boundary through a server-side relay contract that keeps provider credentials out of the desktop client; production hosting, provider invocation, and durable multi-tenant storage remain pending.~~
- [ ] Keep a deterministic fallback provider and actionable unavailable state when the Blackbox relay is disconnected, unavailable, rate-limited, or not configured.
- [x] ~~Separate Orville-managed relay access from user-connected Blackbox access in the provider state and admission contract; durable production identity, billing, and audit storage remain pending.~~
- [ ] Support `blackbox.api_key` only after validating the documented API base URL, endpoint family, model identifiers, request format, streaming behavior, tool-calling behavior, and error schema.
- [!] Add `blackbox.oauth` or device authorization only if Blackbox provides an official third-party flow with documented client registration and token semantics; otherwise do not label API-key entry as "Sign in with Blackbox."
- [x] ~~Define separate provider states: `not_connected`, `connecting`, `connected`, `expired`, `invalid`, `rate_limited`, `unavailable`, and `disabled`.~~
- [ ] Add capability negotiation so chat, streaming, tool calling, multimodal generation, embeddings, Agent API tasks, GitHub operations, and remote task resumption are exposed only when supported by the selected Blackbox endpoint and account plan.
- [ ] Add model discovery with a safe manual-model fallback when the selected Blackbox endpoint does not provide discovery.

- [ ] Add endpoint-family configuration for the standard API, Agent API, and any enterprise or dedicated endpoint instead of assuming one base URL supports every operation.

**User experience:**

- [ ] Make the initial Orville cloud experience usable without requiring the user to enter a Blackbox API key or complete a Blackbox sign-in.
- [ ] Add an explicit `Connect your Blackbox account` action that does not appear as a mandatory onboarding step.
- [ ] Explain that default access is provided through Orville's managed cloud service and is subject to Orville service limits, privacy terms, and availability.
- [ ] If official OAuth/device authorization is confirmed, open the official authorization page, use state/PKCE protections where applicable, validate the callback, and store tokens securely.
- [ ] If official OAuth/device authorization is not confirmed, present `Connect with Blackbox API key` and link to the official dashboard/API-key instructions.
- [ ] Provide connection test, provider/model selection, credential replacement, disconnect, and delete-credential actions.
- [ ] Explain that connecting a Blackbox account may require an eligible subscription and may incur usage charges according to Blackbox's account terms.
- [ ] Ensure disconnecting Blackbox never disables Orville local mode or deletes unrelated task state.
- [ ] Show the active provider, model, endpoint family, privacy mode, and whether the request is local or remote before execution.

**Credential and security controls:**

- [ ] Store Orville-managed Blackbox credentials only on the server-side relay, never in desktop binaries or client configuration.
- [ ] Store user-connected Blackbox API keys, access tokens, and refresh tokens only in the operating system credential store or an equivalent encrypted secret store.
- [ ] Never store Blackbox credentials in `.env` files, project files, task checkpoints, prompts, artifacts, screenshots, crash reports, or source control.
- [ ] Redact `Authorization` headers, token-shaped strings, account identifiers, and sensitive provider errors from logs and telemetry.

- [ ] Do not capture Blackbox browser cookies, reuse undocumented session endpoints, scrape private web application APIs, or bundle a shared Orville-owned Blackbox credential.
- [ ] Add explicit privacy routing controls for local-only, Blackbox-approved, and restricted data.
- [ ] Exclude `.env`, private keys, credential files, and other secret paths from workspace context by default.
- [ ] Add user-visible confirmation before sending workspace files, repository content, images, audio, video, or tool results to Blackbox.
- [ ] Validate TLS, allowed hosts, redirects, callback state, token expiry, and endpoint configuration before use.

**Implementation files and interfaces:**

- [x] ~~Add `ManagedBlackboxRelayAdapter` in `orville_core/providers.py` using the provider-neutral OpenAI-compatible request, response, streaming, and health-check interfaces without accepting a client Blackbox API key.~~
- [x] ~~Extend `orville_core/routing.py` so the default order is explicit user selection, Orville-managed Blackbox cloud relay, user-connected Blackbox provider, other connected providers, local providers when configured, then an actionable unavailable response.~~
- [ ] Extend `orville_core/security.py` with secure credential references, redaction, token lifecycle, and provider-specific permission checks.
- [x] ~~Extend `orville_core/api.py` with managed-relay status, admission, user-account disconnect, and capability-safe endpoints without returning raw credentials; user API-key save/test and official OAuth endpoints remain pending.~~
- [x] ~~Define the initial server-side Orville relay contract for request authorization, quota admission, provider credential isolation, and redacted status; hosted tenant identity, streaming, retries, revocation, and durable audit remain pending.~~
- [ ] Add a complete provider configuration schema containing auth method, endpoint family, base URL, model identifier, account/plan status, capabilities, privacy mode, timeout, and enabled state; the initial relay configuration and redacted status contract are implemented.
- [x] ~~Add `docs/BLACKBOX_INTEGRATION.md` describing supported endpoints, auth modes, setup, privacy behavior, limitations, rollback/disconnect procedures, and the~~

~~standalone relay launcher.~~

- [x] ~~Add cloud relay, relay-server, and API tests with mocked provider forwarding, quota checks, privacy approval, status redaction, client-key rejection, and secret-isolation assertions; live provider and OAuth tests remain pending.~~
- [ ] Extend `test-blackbox-registration.ps1` to cover clean startup, disconnected operation, optional connection, invalid credentials, provider health, disconnect, and local fallback.

**Required validation gates:**

- [x] ~~Cloud relay policy starts without requiring the user to enter a Blackbox credential or complete a Blackbox sign-in; clean packaged-install validation remains pending.~~
- [x] ~~Managed relay provider requests use the configured Orville relay URL and do not send a Blackbox API key from the client; production relay deployment and end-to-end routing remain pending.~~
- [x] ~~The managed relay adapter rejects client-side Blackbox API keys and public relay status explicitly reports `credential_configured: false`; packaged-client inspection remains pending.~~
- [x] ~~User-connected Blackbox access is reported separately from Orville-managed access.~~
- [x] ~~Disconnecting the user's Blackbox account does not disable the default Orville relay in the local API contract; production end-to-end relay validation remains pending.~~
- [ ] Official OAuth/device authentication is not implemented or advertised unless documented by Blackbox and verified end to end.
- [ ] API-key mode succeeds with a valid user-supplied key and fails with an actionable response for 401, 403, 402, 429, timeout, and endpoint errors; storage and redaction are implemented, live provider error mapping remains pending.
- [ ] API-key and token values are absent from logs, checkpoints, artifacts, exceptions, and exported configuration.
- [ ] The provider reports supported and unsupported capabilities before a request is sent; the initial relay contract exposes configured capabilities, while live remote negotiation remains pending.
- [ ] Blackbox failure, expiry, quota exhaustion, or disconnect automatically preserves local execution.

- [ ] Workspace context and remote transmission require the configured privacy policy and user approval.
- [ ] The implementation passes a second-agent security review and a clean-machine integration test.

**Implementation dependency order:**

- [ ] Finalize the Orville-managed cloud relay model, service limits, privacy terms, and tenant authorization.
- [ ] Complete the official Blackbox authentication decision for optional user-account connection and record evidence.
- [ ] Finalize the provider-neutral Blackbox contract and endpoint-family matrix.
- [ ] Implement secure credential references and provider status states.
- [x] ~~Implement the initial standalone server-side Orville relay against documented OpenAI-compatible Blackbox endpoints, with server-only credential injection and mocked forwarding tests; production hosting and live Blackbox verification remain pending.~~
- [x] ~~Implement user API-key connectivity only for explicitly user-connected Blackbox accounts using the protected connector store; live provider health testing remains pending.~~
- [ ] Implement optional OAuth/device flow only if officially supported.
- [ ] Implement capability negotiation, streaming, errors, retries, quota, and rate-limit handling.
- [ ] Integrate managed-relay routing, user-account routing, privacy controls, workspace-context approval, and fallback behavior.
- [ ] Add GUI configuration, connection diagnostics, account connection, disconnect, and credential deletion.
- [x] ~~Execute the current unit and integration validation: 218 tests passed with one existing HTTP-client deprecation warning; clean-install, live quota, production relay, and recovery validation remain pending.~~

## 8.4 Browser and web access

- [ ] Define passive retrieval procedures for informational pages.

- [ ] Define browser takeover procedures for login, CAPTCHA, personal information, and account-specific operations.
- [ ] Define confirmation gates before posting, purchasing, submitting, deleting, or changing account state.
- [ ] Define evidence capture and local preservation for important web findings.
- [ ] Add fallback behavior when a website is unreachable, dynamically rendered, rate-limited, or blocked.

### 8.4 Attached workspace

- [ ] Confirm the intended repository or workspace root before modifying files.
- [ ] Add `AGENTS.md` only when repository-specific instructions are needed and the change is appropriate.
- [ ] Define synchronization rules between sandbox artifacts and the attached Windows workspace.
- [ ] Verify file permissions, line endings, path portability, and executable commands on the target platform.

## 9. Phase 4 — Code Generation and Delivery Pipeline

### 9.1 Planning

- [ ] Convert every implementation request into a written specification.
- [ ] Identify inputs, outputs, interfaces, dependencies, risks, and acceptance tests.
- [ ] Produce a file tree before multi-file implementation.
- [ ] Define the smallest complete vertical slice for early validation.

### 9.2 Implementation

- [ ] Implement source files with clear module boundaries and documentation blocks.
- [ ] Add configuration examples with safe placeholders and explicit environment variables.
- [ ] Add unit tests for core logic.
- [ ] Add integration tests for external boundaries.
- [ ] Add smoke tests for startup, main workflow, and expected failure paths.

- [ ] Add error messages that identify the failed operation without exposing secrets.

### 9.3 Verification

- [ ] Run formatters, linters, type checks, and tests appropriate to the project.
- [ ] Re-run the main workflow from a clean or reproducible environment.
- [ ] Perform independent review against the specification and acceptance criteria.
- [ ] Check generated documentation against the implemented behavior.
- [ ] Record test commands, results, failures, and residual risks.

### 9.4 Delivery

- [ ] Deliver complete source artifacts rather than partial snippets.
- [ ] Include setup, run, test, configuration, deployment, and rollback instructions.
- [ ] Include a concise change summary and known limitations.
- [ ] Attach key supporting files, logs, datasets, visualizations, or generated media when relevant.
- [ ] Preserve the final task graph and execution state for future continuation.

## 12. Phase 5 — Research and Evidence Workflows

- [ ] Define source hierarchy by task type and risk level.
- [ ] Require current-source retrieval for time-sensitive information.
- [ ] Record publication date, access date, source URL, and evidence scope where applicable.
- [ ] Separate primary evidence, secondary reporting, interpretation, and unresolved uncertainty.
- [ ] Add a research synthesis template with executive summary, methodology, findings, limitations, and references.
- [ ] Add a fact-verification checklist for names, dates, numerical values, claims, and quotations.
- [ ] Add a reproducible data-acquisition record for datasets and APIs.

# 13. Phase 6 — Web, Mobile, Media, and Document Workflows

## 11.1 Web and mobile

- [ ] Define project initialization rules for static sites, full-stack web applications, and mobile applications.
- [ ] Define responsive design, accessibility, security, and performance acceptance criteria.
- [ ] Define frontend-backend contracts and environment-specific configuration.
- [ ] Add automated build, test, and preview procedures.
- [ ] Add deployment and rollback instructions.

## 11.2 Image, audio, and video

- [ ] Define asset briefing, generation, editing, licensing, naming, and storage procedures.
- [ ] Define visual verification and media quality checks appropriate to each artifact.
- [ ] Preserve prompts, source assets, generated outputs, and transformation history.
- [ ] Add format, resolution, duration, accessibility, and usage-rights checks.

## 11.3 Documents and presentations

- [ ] Define document templates for reports, specifications, runbooks, and research outputs.
- [ ] Define presentation planning, content validation, design consistency, and export checks.
- [ ] Verify page or slide counts, citations, links, charts, images, and legibility.
- [ ] Preserve editable source formats in addition to exported formats when available.

# 14. Phase 6A — Graphical User Interface

The GUI is a first-class product requirement. It must provide a stylish, intuitive, responsive, accessible, and user-friendly experience for users who want to configure models, describe software requirements, monitor agent execution, review verification results, and retrieve generated artifacts without needing to operate the command line.

## 14.1 Product experience and visual design

- [ ] Define the target users, primary workflows, navigation model, information architecture, and user journeys.
- [ ] Create a cohesive visual design system covering typography, color, spacing, elevation, icons, controls, forms, tables, cards, notifications, dialogs, and empty states.
- [ ] Produce wireframes and high-fidelity mockups before implementation.
- [ ] Define a polished visual style that is professional, modern, consistent, and clear without sacrificing performance or usability.
- [ ] Support light and dark themes, user preference persistence, and clear visual status indicators.
- [ ] Define reusable components and interaction patterns so the interface remains consistent as features expand.

## 14.2 Core GUI workflows

- [ ] Create a dashboard showing active tasks, recent runs, model availability, system health, failures, and generated artifacts.
- [ ] Create a task composer where users can describe software requirements, attach files, define constraints, select models, and specify acceptance criteria.
- [ ] Create a task-plan view showing the generated task graph, dependencies, assigned agents, statuses, blockers, retries, and verification gates.
- [ ] Create an execution monitor with live progress, logs, agent activity, tool calls, elapsed time, and clear pause, resume, retry, and cancel controls.
- [ ] Create a model manager for cloud providers, endpoint-based models, Ollama servers, and imported local model files.
- [ ] Create a model configuration flow accepting user-supplied API credentials or endpoint URLs without exposing secrets in the interface.
- [ ] Create an imported-model workflow for selecting local files or folders, scanning metadata, validating compatibility, activating models, and viewing diagnostics.
- [ ] Create a generation workspace for supported text, code, image, audio, video, vision, embedding, and other modalities based on model capability.
- [ ] Create a verification and review view showing acceptance criteria, test results, source evidence, visual checks, defects, residual risks, and approval state.

- [ ] Create an artifact browser for viewing, downloading, exporting, versioning, and organizing generated code, documents, media, logs, and reports.
- [ ] Create settings for providers, models, privacy routing, storage paths, resource limits, schedules, notifications, and user preferences.
- [ ] Provide contextual help, meaningful error messages, onboarding guidance, tooltips, confirmation dialogs, and recovery actions.

## 14.3 Usability, accessibility, and responsive behavior

- [ ] Make primary workflows understandable without requiring knowledge of agent frameworks, task graphs, or provider-specific APIs.
- [ ] Minimize unnecessary configuration by providing safe defaults while keeping advanced settings available.
- [ ] Use progressive disclosure so complex options do not overwhelm first-time users.
- [ ] Provide keyboard navigation, visible focus states, semantic controls, screen-reader labels, sufficient color contrast, reduced-motion support, and accessible error feedback.
- [ ] Support responsive layouts for desktop, tablet, and smaller screens where the target application permits it.
- [ ] Handle loading, empty, offline, blocked, failed, partial, and long-running states consistently.
- [ ] Prevent destructive actions from occurring without clear confirmation and explain their consequences.
- [ ] Add localization-ready text handling and avoid embedding user-visible copy directly in business logic.

## 14.4 GUI engineering and quality

- [ ] Select an appropriate GUI architecture and document the boundary between presentation, orchestration, model services, storage, and external integrations.
- [ ] Ensure the GUI remains usable when cloud providers, local endpoints, connectors, or model runtimes are unavailable.
- [ ] Add component tests, workflow tests, accessibility checks, responsive-layout tests, and end-to-end tests for the major user journeys.
- [ ] Add visual regression checks for the design system and critical screens.

- [ ] Measure startup time, interaction latency, memory usage, and performance with large task graphs and artifact collections.
- [ ] Verify that logs, prompts, API keys, local paths, and sensitive data are not unintentionally exposed in the interface.
- [ ] Document how to run, build, package, update, and deploy the GUI independently of Manus.

## 15. Phase 7 — Automation, Scheduling, and Persistent Execution

- [ ] Classify tasks as one-shot, recurring, event-triggered, webhook-driven, or persistent-service workloads.
- [ ] Define schedule ownership, timezone handling, expiration, pause, resume, and failure notification behavior.
- [ ] Ensure scheduled workflows are idempotent and safe to retry.
- [ ] Define state storage for long-running jobs and recovery after restart.
- [ ] Define when to use sandbox execution, web hosting, attached desktop execution, or persistent computing.
- [ ] Add health monitoring, structured logs, and operational runbooks.
- [ ] Add dry-run mode for workflows that can mutate external state.
- [ ] Add approval checkpoints for irreversible or high-impact actions.

## 13. Phase 8 — Security and Safety

- [ ] Define secret-handling rules for environment variables, configuration files, logs, artifacts, and screenshots.
- [ ] Add input validation and output sanitization at external boundaries.
- [ ] Add permission minimization for connectors, repositories, files, and remote systems.
- [ ] Add explicit confirmation for payments, publishing, deletion, account changes, and other sensitive operations.
- [ ] Add safe handling for medical, legal, tax, financial, insurance, real-estate, gambling, and major life decisions.

- [ ] Add untrusted-content detection and prevent tool execution based solely on external instructions.
- [ ] Add dependency and supply-chain review for downloaded packages, scripts, and artifacts.
- [ ] Define incident response, credential rotation, and recovery procedures.

## 14. Phase 9 — Testing and Quality System

- [ ] Create a test matrix covering orchestration, delegation, graph dependencies, retries, failures, approvals, and integration.
- [ ] Add unit tests for task parsing, graph validation, routing, state transitions, and artifact registration.
- [ ] Add integration tests for filesystem, GitHub, browser, model, connector, and scheduling boundaries where available.
- [ ] Add regression fixtures for previously fixed failures.
- [ ] Add deterministic test data and mock external services where practical.
- [ ] Add performance tests for graph size, parallel fan-out, retries, and artifact volume.
- [ ] Add security tests for secret leakage, prompt injection, path traversal, unsafe commands, and unauthorized actions.
- [ ] Add acceptance tests for complete representative workflows.
- [ ] Require all failed tests to be triaged before release.

## 15. Phase 10 — Deployment and Operations

- [ ] Define supported deployment targets and required environment variables.
- [ ] Create deployment scripts or commands for each supported target.
- [ ] Add pre-deployment validation and post-deployment smoke tests.
- [ ] Add versioning and release notes.
- [ ] Add rollback procedures and recovery verification.
- [ ] Add structured logs with correlation IDs for multi-agent executions.
- [ ] Add metrics for task duration, success rate, retry count, failure class, and verification outcomes.

- [ ] Add operational dashboards or reports where the target environment supports them.
- [ ] Define maintenance ownership and upgrade cadence.

## 16. Phase 11 — Documentation and User Experience

- [ ] Write a standalone README with prerequisites, installation, configuration, usage, examples, and troubleshooting.
- [ ] Write an architecture document describing agents, graph state, tools, artifacts, and security boundaries.
- [ ] Write an operator runbook for health checks, failures, connector issues, and recovery.
- [ ] Write a contributor guide covering local development, tests, review, and release procedures.
- [ ] Write task templates for research, coding, automation, web development, media, documents, and deployments.
- [ ] Provide examples that run without Manus-specific functionality.
- [ ] Document graceful-degradation behavior when connectors or websites are unavailable.
- [ ] Maintain a glossary for task graph, agent role, artifact, verification gate, connector, and execution state.

## 17. Phase 12 — Continuous Improvement

- [ ] Review completed task graphs for repeated failure patterns.
- [ ] Convert recurring fixes into reusable templates, tests, skills, or automation.
- [ ] Measure time spent in planning, execution, verification, and recovery.
- [ ] Review whether agent assignments match actual task performance.
- [ ] Remove obsolete dependencies, connectors, instructions, and artifacts.
- [ ] Update the readiness report after material environment or architecture changes.
- [ ] Maintain a prioritized backlog with impact, effort, dependencies, and risk.
- [ ] Conduct a quarterly roadmap review or an equivalent milestone review.

## 17A. Final Product Integration Completion Tasks

- [ ] Define the GUI-to-engine API contract for objectives, task graphs, runs, checkpoints, providers, local models, verification records, artifacts, approvals, and event streams.
- [ ] Add an authenticated backend bridge for the GUI with authorization, request validation, CORS policy, rate limits, and redacted audit logging.
- [ ] Connect GUI run creation, pause, resume, cancel, approval, retry, checkpoint, verification, and artifact actions to the engine.
- [ ] Add real-time execution event delivery through a documented polling, SSE, or WebSocket contract.
- [ ] Add model catalog, local-model import, activation, provider health, and routing controls to the GUI.
- [ ] Add artifact storage, preview, download, versioning, and retention controls.
- [ ] Add persistent observability traces, metrics, evaluation fixtures, security regression tests, and release thresholds.
- [ ] Add packaging, installation, configuration migration, upgrade, rollback, and deployment workflows for standalone use.
- [ ] Validate the complete product in a clean environment with configured cloud, local endpoint, and no-provider fallback scenarios.

# 18. Prior-Phase Audit and Gap Register

This audit compares the roadmap with the implementation artifacts currently present in the workspace: `orville_core/`, `tests/`, the runnable example, and the current documentation set. A checked item means the narrow behavior is implemented and tested; a broader roadmap item remains unchecked when only a partial slice exists.

## 18.1 Confirmed completed slices

- [x] ~~Phase 1 has a typed task graph, dependency validation, synchronous execution, structured events, atomic JSON checkpoints, failure blocking, resume behavior, a runnable example, and regression tests.~~
- [x] ~~Phase 2 has provider-neutral configuration, Gemini generation, Ollama generation, custom Ollama-compatible endpoints, health checks, structured-output request construction, tool-call normalization, a local model catalog, and redacted configuration metadata.~~

- [x] ~~Phase 3 has normalized stream chunks, NDJSON/SSE parsing, Gemini and Ollama streaming, multimodal request conversion, Gemini and Ollama embeddings, capability-aware selection, local-only filtering, fallback for complete responses and embeddings, and endpoint preflight validation.~~
- [x] ~~The current automated regression suite contains 21 passing tests and compilation succeeds for package, tests, and examples.~~

## 18.2 Missed or incomplete prerequisites from earlier phases

| Gap | Affected roadmap area | Current evidence | Required action | Priority |
|---|---|---|---|---|
| Project state files are missing | Phase 0 governance | Only `TODO.md` is present; no `PROJECT.md`, `STATE.md`, or `TASK_GRAPH.md` | Create durable project, execution-state, and graph records | P0 |
| Agent delegation contracts are not operational | Phase 1 delegation and Phase 2 agent contracts | No routing schema for specialist agents or formal handoff objects exists | Define agent registry, handoff envelope, ownership, conflict rules, and verification assignment | P0 |
| Task intake is not implemented | Phase 1 task intake | Engine accepts a prebuilt `TaskGraph`; it does not normalize user specifications | Add intake schema, objective classification, assumptions, clarification gates, and acceptance criteria generation | P0 |
| Parallel, conditional, approval, and human-in-the-loop execution is missing | Phase 1 graph construction | Current engine is synchronous and has no approval or conditional node model | Extend graph node types, scheduler, approval pauses, cancellation, and safe parallel execution | P0 |

| | | | | |
|---|---|---|---|---|
| Output verification is not independent | Verification Agent requirements | Task completion is currently treated as verification by the same execution path | Add independent verifier tasks and acceptance-gate results separate from task execution | P0 |
| Provider routing is not integrated with graph tasks | Phase 2 and Phase 3 integration | Router is a standalone API; no model-backed task handler or persisted routing metadata exists | Add model task handler, provider/model metadata, retry policy, and checkpoint integration | P0 |
| Local model activation is only cataloging | Imported model requirements | Files can be hashed and registered, but runtime validation and activation are not implemented | Add safe runtime adapters, resource checks, lifecycle operations, and generation smoke tests | P1 |
| GUI is not started | GUI phase | No frontend or desktop application exists | Select an application architecture, build the shell, and connect task/model/execution/artifact workflows | P1 |
| Security controls remain mostly documented requirements | Security phases | Endpoint preflight exists, but sandboxing, tool policy, secret storage, prompt-injection defenses, and audit controls are absent | Implement threat model and least-privilege enforcement before external actions | P0 |
| Runtime and connector health checks are not packaged | Environment reliability | Initialization inspected the environment manually; no repeatable project command exists | Add health-check command and connector diagnostics | P1 |

| | | | | |
|---|---|---|---|---|
| Production-quality testing is not established | Testing and quality | Unit-style fake-transport tests exist; no integration matrix, acceptance harness, security suite, or performance suite exists | Add layered test strategy and release gates | P1 |
| Standalone product documentation is incomplete | Documentation and user experience | Component documents exist; README, runbook, contributor guide, templates, and glossary are missing | Complete the standalone documentation set | P1 |
| Deployment and operations are not implemented | Deployment and operations | No deployment target, packaging, metrics, tracing export, rollback, or maintenance process exists | Define deployment architecture and operational lifecycle | P1 |

## 18.3 Roadmap consistency corrections

- [x] ~~Replace nonstandard `[done]` markers with the defined `[x]` marker.~~
- [ ] Renumber and normalize duplicated or inconsistent phase headings before the roadmap becomes the source for automated task generation.
- [ ] Split broad phase labels from implementation increments so Phase 2 provider work and Phase 3 media work are not conflated.
- [ ] Add a machine-readable task identifier to every backlog item.
- [ ] Add an explicit status, owner, dependency, acceptance test, and artifact reference to every priority item.

## 18.4 Recommended next order

1. Create `PROJECT.md`, `STATE.md`, and `TASK_GRAPH.md`.

2. Implement normalized task intake and agent handoff contracts.
3. Integrate provider routing with model-backed task handlers and checkpoint metadata.
4. Extend the scheduler for parallel, conditional, approval, cancellation, and independent verification nodes.
5. Implement local-model activation safely before exposing imported models as usable generation targets.
6. Establish the GUI foundation after the orchestration and provider interfaces stabilize.
7. Add security enforcement, acceptance evaluation, observability, packaging, and deployment gates.

# 19. Priority Backlog

| Priority | Task | Owner | Dependency | Acceptance criterion |
|---|---|---|---|---|
| [x] | Create `STATE.md`, `TASK_GRAPH.md`, and `PROJECT.md` | Orchestration Agent | None | Project state and graph can be resumed from files |
| P0 | Repair or replace `python fast api` connector configuration | Automation Agent | Connector inspection | Tool discovery succeeds without URL parsing errors |
| P0 | Investigate `fly dev` timeout and define fallback | Automation Agent | Endpoint diagnosis | Either a harmless capability call succeeds or a documented fallback is active |
| [x] | Define task intake and graph schemas | Orchestration Agent | Governance files | A software objective can be normalized into a validated graph skeleton |
| [x] | Implement first orchestration and checkpointing vertical slice | Code Synthesis Agent | Graph schema | Five automated tests pass and a runnable example persists a checkpoint |

| | | | | |
|---|---|---|---|---|
| [x] | Create verification checklist and defect format | Verification Agent | Agent contracts | Verification records and deterministic acceptance checks are available |
| [x] | Implement routing and delegation rules | Orchestration Agent | Intake and graph schemas | Agent registry selection and model-backed task routing are available |
| [x] | Add persistent execution state and recovery | Orchestration Agent | Graph state model | Project state files and checkpoint-backed workflow state are available; distributed recovery remains pending |
| P1 | Create representative end-to-end workflow tests | Verification Agent | Routing and state | Research, coding, and automation scenarios pass acceptance tests |
| [x] | Implement execution controls | Orchestration Agent and Automation Agent | Task and checkpoint contracts | Conditions, approvals, cancellation, timeouts, idempotency, ownership checks, and 34 passing tests are available |
| [x] | Implement safe in-process parallel scheduling | Orchestration Agent and Pair Programmer Agent | Task and checkpoint contracts | Independent tasks run with isolated context snapshots and serial checkpoint reconciliation; 36 tests pass |
| [x] | Implement provider- | Code Synthesis Agent | Model-provider schema | Gemini, Ollama, and a custom local endpoint |

| | | | | |
|---|---|---|---|---|
| | agnostic model adapters | | | are configured and health-checked without changing orchestration code |
| [x] | Implement streaming, multimodal, and embedding adapters | Code Synthesis Agent | Provider-neutral request and response contracts | Gemini and Ollama-compatible adapters normalize streams, media parts, and embeddings with 16 tests passing |
| [x] | Implement capability-aware provider routing and endpoint validation | Orchestration Agent and Code Synthesis Agent | Provider registry and capability contracts | Eligible providers are selected by capability, preference, and local-only policy, with controlled fallback and 21 tests passing |
| [x] | Implement imported local model catalog and activation baseline | Code Synthesis Agent | Local model schema and runtime detection | Import, hash, inspect, validate, activate, deactivate, and provider bridge are available; hardware/runtime isolation remains pending |
| P1 | Design and implement the GUI foundation | IDE Agent and Prototype Agent | Stable orchestration and model interfaces | A user can create a task, configure a model, monitor execution, review verification, and access artifacts through a polished interface |
| P1 | Implement core GUI workflows | Code Synthesis Agent and Prototype Agent | GUI foundation | Dashboard, task composer, graph view, execution monitor, model manager, |

| | | | | |
|---|---|---|---|---|
| | | | | verification view, and artifact browser pass workflow tests |
| P1 | Publish standalone installation and operation documentation | IDE Agent | Stable architecture | A user can run the system outside Manus using the documentation |
| P2 | Add metrics, logs, and operational dashboards | Automation Agent | Persistent state | Execution health can be reviewed after completion |
| [x] | Add provider health discovery and circuit breaking baseline | Automation Agent | Provider registry | Capability discovery and in-process failure suppression are tested and documented |
| P2 | Add security and prompt-injection regression suite | Research and Verification Agents | External-boundary rules | Unsafe instructions are not executed from untrusted content |
| P2 | Add reusable task templates and project starters | Prototype Agent | Stable contracts | Common objectives can begin from validated templates |
| P1 | Implement durable checkpoints, replay, and resumable execution | Orchestration Agent | Graph state model | Interrupted workflows resume with an auditable state history |
| P1 | Implement threat model and least-privilege tool controls | Automation Agent and Verification Agent | Security boundaries | Unsafe tool access and external actions are blocked or require approval |
| P1 | Implement OpenTelemetry- | Automation Agent | Event model | Model, agent, tool, approval, |

| | | | | |
|---|---|---|---|---|
| | compatible execution tracing | | | retry, and artifact telemetry is queryable without leaking sensitive content |
| P1 | Create realistic repository-level coding evaluations | Verification Agent | Isolated execution harness | Generated patches are tested behaviorally across representative repositories |
| P1 | Establish WCAG 2.2 GUI release gates | IDE Agent and Verification Agent | GUI foundation | Critical workflows pass keyboard, focus, contrast, reflow, screen-reader, and status-message checks |
| P2 | Implement model provenance and safe-serialization verification | Code Synthesis Agent | Local model catalog | Imported models carry checksum, license, provenance, format, and activation evidence |

## 19. Standard Execution Record Template

Use the following record for each future objective:

Markdown

```
# Execution Record: <objective title>

- Task ID:
- Objective:
- Requesting user:
- Start time:
- Target environment:
- Deliverables:
- Constraints:
- Assumptions:
```

```
- Risk level:
- Required connectors:
- Required skills:
- Acceptance criteria:

## Task Graph

| ID | Task | Agent | Depends on | Status | Validation |
|---|---|---|---|---|---|
| T1 |  |  |  | planned |  |

## Execution Log

| Time | Task ID | Event | Result | Artifact or reference |
|---|---|---|---|---|
|  |  |  |  |  |

## Verification

- [ ] Requirements checked.
- [ ] Outputs inspected.
- [ ] Tests or validation commands executed.
- [ ] Independent review completed.
- [ ] Known limitations recorded.
- [ ] Final artifacts integrated and delivered.

## Final Outcome

- Result:
- Artifacts:
- Tests:
- Remaining risks:
- Recommended next action:
```

## 20. Immediate Next Execution Sequence

1. **Create project state files.** The Orchestration Agent should establish `PROJECT.md`, `STATE.md`, and `TASK_GRAPH.md` using this roadmap as the initial source of truth.
2. **Repair integration reliability.** The Automation Agent should inspect and correct the `python fast api` connector and investigate the `fly dev` timeout without enabling unrelated services.
3. **Define executable schemas.** The Code Synthesis Agent should implement or document the task intake, task node, graph state, agent handoff, artifact registry, verification record, and model-provider configuration schemas.

4. **Implement provider adapters.** The Code Synthesis Agent should add a provider-neutral interface with Gemini, Ollama, and OpenAI-compatible endpoint examples using user-supplied credentials or URLs.
5. **Implement local model import.** The Code Synthesis Agent and Prototype Agent should add import, cataloging, metadata detection, checksum validation, resource checks, runtime activation, and supported generation using user-downloaded model files.
6. **Design the GUI foundation.** The IDE Agent and Prototype Agent should establish the design system, navigation, responsive layout, accessibility baseline, and application shell for the main user journeys.
7. **Implement the core GUI workflows.** The Code Synthesis Agent should connect task creation, model management, imported-model activation, task-graph monitoring, verification review, and artifact delivery to the interface.
8. **Implement the smallest vertical slice.** The Prototype Agent should create a local workflow that accepts one objective, constructs a graph, selects a configured cloud, endpoint, or imported local model, assigns roles, records state, and emits a verified report.
9. **Independently verify the slice.** The Verification Agent should test normal execution, missing-input handling, dependency failure, retry behavior, provider fallback, local-only routing, imported-model activation, unsupported formats, insufficient resources, invalid credentials, unreachable endpoints, and final artifact delivery.
10. **Integrate and document.** The IDE Agent and Orchestration Agent should consolidate the implementation, update documentation, and record the completed state.

# 21. Research-Derived Hardening and Evaluation Requirements

The following requirements were added after reviewing primary documentation and research from LangGraph, Ollama, Hugging Face Safetensors, OWASP, NIST, W3C WCAG, OpenTelemetry, Model Context Protocol, and SWE-bench. They convert the findings into implementation and release tasks rather than treating the research as general guidance.

## 21.1 Orchestration reliability

- [ ] Separate deterministic workflow steps from agentic steps and require deterministic implementations for safety-critical, authorization, validation, persistence, and artifact-integrity operations.

- [ ] Implement durable checkpoints before and after material agent, tool, model, approval, and artifact operations.
- [ ] Support replay, resume, pause, cancellation, retry, and controlled state inspection after interruption or failure.
- [ ] Stream graph, agent, tool, model, approval, and artifact events to the GUI and persist an auditable event history.
- [ ] Define short-term task memory, long-term project memory, retention, deletion, isolation, and user-editing rules.
- [ ] Add idempotency keys and deduplication for external actions, retries, scheduled jobs, and artifact writes.

## 21.2 Model and local-file security

- [ ] Prefer Safetensors or another safe serialization format where supported and classify unsafe formats before import.
- [ ] Isolate model conversion, metadata inspection, loading, and execution from the host system with least privilege.
- [ ] Never execute arbitrary code, scripts, hooks, or model-provided commands merely because they are present in an imported model directory.
- [ ] Verify checksums, provenance, license metadata, source information, and optional signatures or attestations before activation.
- [ ] Distinguish full models, adapters, quantized models, tokenizers, configuration files, and auxiliary assets in the catalog.
- [ ] Require base-model identity and compatibility checks when importing adapters; provide a clear diagnostic when the base model does not match.
- [ ] Add resource-aware scheduling for CPU, RAM, GPU, VRAM, disk, context length, concurrency, and thermal or power constraints where available.

## 21.3 Provider and MCP security

- [ ] Create a threat model covering prompt injection, excessive agency, insecure output handling, sensitive information disclosure, supply-chain risk, context poisoning, and unbounded tool access.

- [ ] Apply least-privilege permissions, tool allowlists, filesystem boundaries, network egress policies, and per-task credentials.
- [ ] Add prompt and output boundary markers, untrusted-content labels, and explicit separation between instructions, retrieved data, tool results, and user approvals.
- [ ] Validate remote endpoint schemes, hosts, ports, redirects, authorization servers, and localhost callback handling to reduce SSRF and OAuth risks.
- [ ] Prevent token passthrough and confused-deputy behavior by binding credentials to the intended provider, user, task, and scope.
- [ ] Protect MCP state handles from fixation, replay, cross-user access, and tampering.
- [ ] Add authorization-decision logs without storing secret values or unnecessary sensitive content.
- [ ] Add dry-run and approval modes for every external action that can publish, delete, purchase, transfer, modify accounts, or change production state.

## 21.4 Evaluation and observability

- [ ] Define task-specific evaluation datasets and golden cases for planning, code generation, debugging, refactoring, research, GUI workflows, and model import.
- [ ] Evaluate generated software in isolated, reproducible environments using tests and behavioral acceptance criteria rather than text similarity alone.
- [ ] Add repository-level coding evaluations using realistic issues, patches, dependency installation, test execution, and regression checks.
- [ ] Track per-run model/provider/version, prompt or prompt hash according to privacy policy, tool calls, agent handoffs, retries, approvals, artifacts, latency, token usage, finish reasons, cache use, cost metadata, and failures.
- [ ] Implement OpenTelemetry-compatible traces, metrics, and events for graph nodes, agents, model calls, tool calls, MCP calls, approvals, and artifact operations.
- [ ] Make capture of prompts, completions, tool arguments, and tool results explicitly opt-in, redacted, access-controlled, and retention-limited.
- [ ] Add trace comparison across runs to identify nondeterminism, regressions, repeated failure patterns, and unexpected tool behavior.

- [ ] Define release thresholds for task success, test pass rate, safety violations, latency, cost, failure recovery, and GUI accessibility.

### 21.5 GUI quality gates

- [ ] Use WCAG 2.2 as the accessibility baseline for keyboard operation, visible and unobscured focus, logical focus order, no keyboard traps, contrast, reflow, status messages, and error feedback.
- [ ] Test the GUI with keyboard-only navigation, screen readers, high zoom, reduced motion, high contrast, small screens, slow connections, and long-running operations.
- [ ] Ensure live execution updates are communicated through accessible status messages and are not conveyed by color alone.
- [ ] Add usability testing with first-time users for task creation, model setup, local model import, execution review, verification review, and artifact export.
- [ ] Add visual regression tests for the design system, critical states, theme variants, loading states, errors, partial failures, and approval dialogs.

### 21.6 Lifecycle governance

- [ ] Maintain a risk register with risk owner, affected asset, likelihood, impact, mitigation, residual risk, review date, and evidence.
- [ ] Define pre-release, post-release, incident-triggered, and periodic evaluation procedures.
- [ ] Document model, provider, connector, prompt, tool, dependency, and GUI version changes for reproducibility.
- [ ] Define incident response for model failures, data leakage, unsafe tool use, compromised connectors, corrupted model files, and production regressions.
- [ ] Add a deprecation and migration process for providers, model formats, APIs, MCP versions, runtime dependencies, and GUI components.

## 22. Research Sources Used

- LangGraph overview — durable execution, stateful graphs, deterministic and agentic steps, persistence, streaming, human-in-the-loop, memory, debugging, and deployment.

- Ollama importing a model — Safetensors and GGUF imports, Modelfiles, adapters, base-model compatibility, model creation, and test runs.
- Hugging Face Safetensors — safe tensor serialization, zero-copy loading, and partial tensor loading.
- OWASP GenAI Security Project — LLM and agentic security risks.
- NIST Generative AI Profile — lifecycle risk management and trustworthy AI evaluation.
- W3C WCAG 2.2 — keyboard accessibility, focus, contrast, reflow, and robust interface requirements.
- OpenTelemetry GenAI observability — model, token, finish-reason, prompt, completion, and tool telemetry.
- MCP Security Best Practices — confused deputy, token passthrough, SSRF, state-handle, OAuth, local server, and scope controls.
- SWE-bench overview — realistic repository-level software tasks and reproducible Docker-based evaluation.

## Expanded Integrations and Provider Coverage

- [x] ~~Audit why the Integrations workspace is not visible or reachable in the live preview.~~
- [x] ~~Research current reputable hosted LLM API patterns and Stable Horde text-generation support.~~
- [x] ~~Add a clearly visible Integrations entry point while preserving the Signal Room design.~~
- [x] ~~Add provider presets for Gemini, OpenAI, Anthropic, OpenRouter, Cohere, Mistral, Groq, Together, DeepSeek, xAI, Perplexity, Azure OpenAI, Amazon Bedrock-compatible gateways, and local runtimes where the API contract is supported.~~
- [x] ~~Add a generic OpenAI-compatible API/local URL configuration path with custom headers and endpoint support.~~
- [x] ~~Add Stable Horde text/code-generation support, explicitly excluding image-generation workflows.~~
- [x] ~~Verify provider registration, redacted credentials, health checks, routing, generation, and actionable failures.~~
- [x] ~~Save a verified preview checkpoint and report exact provider setup steps.~~

## 23. Roadmap Completion Gate

The roadmap may be marked complete only after the system demonstrates at least one end-to-end workflow that starts from a user objective and finishes with a task graph, delegated execution, independent verification, persisted state, runnable artifacts, documentation, and a final delivery summary. Any unavailable connector or external service must have a tested fallback or an explicit operational limitation.

## Current Integration Status — Finalization Pass

- [x] ~~Security policy baseline implemented: tool allowlists, filesystem boundaries, network allowlists, secret redaction, and external-action guards.~~
- [x] ~~Local model lifecycle baseline implemented: validation, activation, deactivation, catalog removal, and guarded deletion.~~
- [x] ~~Provider hardening baseline implemented: capability discovery, health-aware exclusion, and circuit breaking.~~
- [x] ~~GUI foundation implemented in the separate `orville_gui` project with Signal Desk design, task composer, model catalog, local-model import interaction, evidence ledger, and responsive layouts.~~
- [x] ~~Initial authenticated GUI API bridge implemented in `orville_core/api.py` with objective intake, checkpoint retrieval, persisted event retrieval, cancellation, approval, project state, and health routes.~~
- [x] ~~GUI API client implemented with in-memory bearer-token handling and objective/event requests.~~
- [ ] Production GUI bridge remains pending: durable identity and authorization scopes, CORS and rate-limit enforcement, backend run manager injection, artifact APIs, SSE/WebSocket push, and database-backed state.
- [ ] Final operational pass remains pending: OpenTelemetry export, evaluation harness, security regression suite, packaging and migration workflows, deployment configuration, rollback procedures, and clean-environment acceptance testing.

## Finalization Pass Status

- [x] ~~Add authenticated FastAPI bridge baseline with objective intake, executable-run injection, run state, persisted events, approvals, cancellation, project state, health, CORS configuration, and in-memory rate limiting.~~

- [x] ~~Add root-bound artifact registry with SHA-256 metadata, MIME detection, traversal protection, authenticated listing, and retrieval routes.~~
- [x] ~~Add redacted JSONL trace recording and deterministic acceptance evaluation primitives.~~
- [x] ~~Add standalone README, API bridge guide, observability/evaluation guide, and package API extra metadata.~~
- [x] ~~Add GUI API client primitives for health, objective creation, run state, events, cancellation, approvals, artifact listing, and artifact URLs; bearer tokens remain in memory only.~~
- [x] ~~Verify the backend with compilation and 53 tests passing, with 3 API tests skipped only when FastAPI test extras are unavailable.~~
- [x] ~~Verify the GUI with TypeScript checking, production build, desktop preview, and mobile preview.~~
- [ ] Replace in-memory API graphs and rate limits with durable database-backed state and distributed rate limiting.
- [ ] Provide a production identity provider, scoped authorization, TLS, deployment secrets, CORS allowlist, and audit-log sink.
- [ ] Inject real model-backed handlers and run manager into the deployed API; no fake generation handlers are permitted.
- [ ] Implement SSE/WebSocket event push, full local-model runtime activation, sandboxed process execution, and hardware-aware resource checks.
- [ ] Complete production acceptance, security, accessibility, performance, repository-level code-generation evaluation, packaging, deployment, rollback, and disaster-recovery tests.

## Roadmap Completion Pass — Verified Update

- [x] ~~Add durable SQLite checkpoint persistence with restart-compatible load, list, and existence operations.~~
- [x] ~~Make the authenticated API use SQLite by default, with explicit JSON compatibility mode.~~

- [x] ~~Persist objectives at creation time so they remain executable after API process recreation.~~
- [x] ~~Add authenticated SSE event replay/streaming with stable event IDs and reconnect cursor.~~
- [x] ~~Add standalone `orville` CLI commands for health, run listing, and checkpoint inspection.~~
- [x] ~~Add Windows-safe SQLite connection cleanup and regression tests.~~
- [x] ~~Verify backend compilation, 55 tests passing with 3 optional API skips, package installation smoke test, and CLI health.~~
- [x] ~~Verify GUI TypeScript checking and production build after API-client event-stream support.~~
- [ ] Connect deployed GUI to a configured backend URL and authenticated token through a server-side secret mechanism.
- [ ] Complete external identity, scoped authorization, TLS, distributed rate limiting, and durable audit logging.
- [ ] Complete real model runtime injection, sandboxed local process execution, resource validation, and provider capability negotiation.
- [ ] Run clean-environment acceptance and deployment validation after choosing the target hosting topology.

## Instruction-First Agentic Code Completion

- [ ] Audit the current opening screen and objective composer flow.
- [ ] Make the initial view request agentic code-completion instructions before showing the workspace.
- [ ] Add clear fields for task instructions, repository or project context, expected changes, and acceptance criteria.
- [ ] Launch the created agentic run directly into the live code-generation viewer.
- [ ] Preserve access to the Signal Room workspace, Integrations, runs, artifacts, state, events, and API docs.
- [ ] Verify empty, validation, success, error, and reconnect states.

- [ ] Save a verified preview checkpoint.

## Replit/Base44-Style Feature Expansion Audit

- [x] ~~Inventory which researched platform capabilities are already present in Orville and which remain missing.~~
- [x] ~~Research current official feature descriptions for Replit, Base44, Cursor, CrewAI, and LangGraph.~~
- [x] ~~Create a normalized feature matrix with implementation status, standalone-Windows fit, dependencies, and verification criteria.~~
- [x] ~~Prioritize agent workspace, repository context, file operations, terminal execution, checkpoints, approvals, collaboration, automations, integrations, observability, and packaging features.~~
- [x] ~~Implement the selected core feature slice without removing existing Signal Room functionality.~~
- [x] ~~Verify backend, frontend, provider routing, safety controls, and packaging readiness baseline.~~
- [x] ~~Save an implementation checkpoint and deliver the remaining roadmap explicitly.~~

## Repository-Aware Code Completion

- [x] ~~Audit current workspace, security, project, artifact, and execution contracts.~~
- [x] ~~Define safe repository selection, indexing, diff, command, and self-repair contracts.~~
- [x] ~~Implement secure repository-folder selection and indexed file-context APIs.~~
- [x] ~~Implement reviewable diffs, allowlisted terminal execution, and bounded self-repair iterations.~~
- [x] ~~Integrate repository controls into the Agent workspace and instruction-first intake.~~
- [x] ~~Verify path boundaries, secret redaction, command allowlists, diff safety, repair limits, frontend build, and end-to-end execution.~~
- [ ] Save a repository-aware checkpoint.

## Restored Orville Product Shell

- [ ] Audit current navigation, intake, task history, projects, settings, personal agent, and persistence behavior.
- [ ] Re-read runtime, automation, hosting, and full-stack guidance for always-on personal-agent boundaries.
- [ ] Define the sidebar information architecture and first-screen interaction model.
- [ ] Restore a collapsible sidebar with New Task, Personal Agent, Projects, task history, and Settings.
- [ ] Make the main screen a friendly instruction-first New Task input window.
- [ ] Add a Personal Agent workspace with isolated runtime status and persistent memory controls.
- [ ] Add clickable Projects and previous-task recovery flows.
- [ ] Preserve provider configuration, repository tools, live code viewer, runs, artifacts, state, events, and API docs.
- [ ] Verify desktop, collapsed-sidebar, and responsive behavior plus task recovery and settings flows.
- [ ] Save a verified restored-shell checkpoint.

## Restored Orville Shell Status

- [x] ~~Audited current navigation, intake, task history, projects, settings, personal agent, and persistence behavior.~~
- [x] ~~Re-read runtime, automation, hosting, and static frontend guidance for the always-on local-agent boundary.~~
- [x] ~~Defined the sidebar information architecture and first-screen interaction model.~~
- [x] ~~Restored a collapsible sidebar with New Task, Personal Agent, Projects, task history, Settings, and preserved Operations menus.~~
- [x] ~~Made the main screen a friendly instruction-first New Task input window.~~
- [x] ~~Added Personal Agent status, local Windows runtime metadata, persistent project memory, pause/resume controls, and project-scoped memory APIs.~~
- [x] ~~Added Projects and previous-task recovery APIs and views.~~

- [x] ~~Preserved provider configuration, repository tools, live code viewer, runs, artifacts, state, events, and API docs.~~
- [x] ~~Verified frontend build, 99 backend tests, control-plane API regression, and desktop preview rendering.~~
- [ ] Save a verified restored-shell checkpoint.

## Runs Walkthrough Video

- [ ] Define the complete run lifecycle narrative and scene order.
- [ ] Prepare Signal Room visual references and instructional overlays.
- [ ] Generate a walkthrough covering intake, planning, provider generation, live code, verification, approvals, artifacts, failure, and repair.
- [ ] Review the video for readable labels, sequence completeness, and factual alignment with the current Orville implementation.
- [ ] Deliver the final video artifact.

## Runs Walkthrough Video Status

The instructional walkthrough was rendered as `/home/ubuntu/orville-runs-walkthrough.mp4` after the AI video generator reported that the free-plan daily quota had been reached. The fallback is a 30-second, 1280×720 H.264 video with six readable stages: lifecycle overview, New Task intake, dependency-aware planning, live streamed implementation, verification/approval/repair, and completed artifacts. `ffprobe` confirmed valid MP4 integrity, 30-second duration, 1280×720 dimensions, and H.264 encoding.

## Broad Manus-Like Capability Expansion

- [ ] Audit existing Orville capabilities against research, browser, coding, workspace, memory, artifact, automation, connector, scheduling, notification, deployment, and observability categories.
- [ ] Define standalone Windows equivalents and explicitly document proprietary Manus capabilities that cannot be reproduced literally.
- [ ] Implement the highest-value missing agent runtime, browser/research, workspace, memory, approval, and automation foundations.

- [ ] Implement document, spreadsheet, presentation, data, media, and code artifact workflows.
- [ ] Implement connectors, schedules, notifications, deployment helpers, and observability equivalents.
- [ ] Integrate new capabilities into Signal Room without removing existing menus or workflows.
- [ ] Verify security, compatibility, end-to-end behavior, clean-host operation, and executable packaging.
- [ ] Save a broad-capability checkpoint and deliver a parity report.

## Safe Browser Session Adapter

- [ ] Audit current browser adapter, security policy, API initialization, and GUI capability status.
- [ ] Define browser session lifecycle, domain allowlist, takeover, approval, and audit contracts.
- [ ] Implement a local browser-session adapter with read-only defaults and fail-closed navigation.
- [ ] Add authenticated session, allowlist, navigation, takeover, approval, and audit routes.
- [ ] Integrate browser controls and takeover prompts into Signal Room.
- [ ] Verify blocked domains, allowed navigation, sensitive-action approval, takeover state, audit records, and responsive UI behavior.
- [ ] Save a browser-session adapter checkpoint.

## Browser Actions, Recovery, and Run Citations

- [ ] Audit browser sessions, action state, run events, artifact storage, and shutdown lifecycle.
- [ ] Define approval records for form submissions and file downloads with redaction rules.
- [ ] Record browser actions and implement approval-gated form submission and download operations.
- [ ] Persist browser sessions and recover interrupted sessions safely after restart or shutdown.

- [ ] Extract page titles, readable text, metadata, and downloaded-source references.
- [ ] Attach source records and citations to agent runs and generated artifacts.
- [ ] Integrate action approvals, recovery state, and citations into Signal Room.
- [ ] Verify security, persistence, clean shutdown, run linkage, and frontend behavior.
- [ ] Save a verified browser workflow expansion checkpoint.

## Current execution — Windows release validation

- [x] ~~Read the persistent-computing and automation-and-scheduling guidance.~~
- [x] ~~Inspect research execution and citation persistence paths.~~
- [x] ~~Implement automatic citation capture for active research-agent stages.~~
- [x] ~~Add local fixture site and end-to-end browser approval smoke test.~~
- [x] ~~Inspect current Windows packaging scripts and entrypoints.~~
- [x] ~~Build the Windows executable with all runtime assets.~~
- [x] ~~Validate clean-machine startup, API health, and GUI launch behavior.~~
- [x] ~~Run the complete regression suite and record artifacts.~~
- [ ] Save a final project checkpoint and deliver the executable path and validation results.

## Current execution — checkpoint and safe cleanup

- [x] ~~Save the current project checkpoint before cleanup.~~
- [x] ~~Inventory generated versions and release assets in the attached Projects directory.~~
- [x] ~~Confirm the exact deletion scope before irreversible removal.~~
- [x] ~~Remove only confirmed obsolete versions.~~
- [x] ~~Validate the preserved current release and report the cleanup results.~~

## Current execution — Stable Horde multimodality upgrade

- [x] ~~Inspect Stable Horde provider contracts, current adapter behavior, and routing capability flags.~~
- [x] ~~Verify current Stable Horde API modality support and asynchronous request semantics.~~

- [x] ~~Implement capability-aware text, code, and image request paths; reject unsupported Stable Horde video requests safely.~~
- [x] ~~Expose modality configuration and validation in the API and Signal Room GUI.~~
- [x] ~~Add mocked provider, routing, security, and frontend build tests.~~
- [ ] Save a checkpoint and document supported capabilities and limitations.

## Current execution — Hugging Face provider integration

- [x] ~~Inspect current provider, media, and integrations architecture.~~
- [x] ~~Implement Hugging Face hosted text/code routing and capability metadata.~~
- [x] ~~Add Hugging Face image/video request adapters with fail-closed capability checks.~~
- [x] ~~Expose Hugging Face configuration and media controls in the Signal Room.~~
- [x] ~~Add mocked provider, API, security, capability, and frontend build tests.~~
- [x] ~~Rebuild and clean-start validate the Windows executable with Hugging Face dependencies.~~
- [x] ~~Save a checkpoint and document Hugging Face setup and limitations.~~

## Current execution — Hugging Face Hub model browser

- [x] ~~Inspect the local-model catalog, API routes, packaging assumptions, and provider UI.~~
- [x] ~~Implement machine capability detection with conservative support heuristics.~~
- [x] ~~Add Hugging Face Hub search and model metadata retrieval with pagination and safe filters.~~
- [x] ~~Add guarded model downloads with size, license, checksum, path, and runtime validation.~~
- [x] ~~Register downloaded models in the local catalog without executing repository code.~~
- [x] ~~Add Signal Room search, model cards, capability details, and a supported-only toggle.~~
- [x] ~~Run API, security, catalog, frontend, and Windows packaging validation.~~
- [ ] Save a checkpoint and document model-browser usage and limitations.

## Current execution — resumable Hub downloads and runtime compatibility

- [x] ~~Inspect current Hub download flow, local catalog, and runtime configuration contracts.~~
- [x] ~~Implement durable download-job records with resumable state, progress, cancellation, and restart recovery.~~
- [x] ~~Preserve explicit approval, path containment, size limits, checksums, and untrusted-repository handling.~~
- [x] ~~Add runtime-specific checks for Ollama, llama.cpp, and Transformers.~~
- [x] ~~Add API routes for download jobs, progress, cancellation, and compatibility reports.~~
- [x] ~~Add Signal Room progress cards, cancel controls, runtime selection, and compatibility results.~~
- [x] ~~Run backend, security, frontend, packaging, and clean Windows executable validation.~~
- [ ] Save a checkpoint and document download/runtime behavior and limitations.

## Current execution — pausable downloads and local generation models

- [ ] Inspect download jobs, local catalog, provider routing, and Integrations selection contracts.
- [ ] Implement pause/resume state with cooperative worker control and restart recovery.
- [ ] Connect installed local models to runtime validation, activation, and provider routing.
- [ ] Add local models to provider and New Task generation selectors.
- [ ] Add pause/resume, activate, and select controls to the Signal Room.
- [ ] Run model-installation, routing, security, frontend, packaging, and Windows validation.
- [ ] Save a checkpoint and document the local-model workflow.

## Current Task — Hub Transfer Retry and Backoff Telemetry

- [ ] Inspect durable Hub download failure and persistence flow.
- [ ] Add bounded retry policy with exponential backoff and cooperative pause/cancel handling.
- [ ] Persist retry counters, next retry timing, last error, and transfer telemetry.
- [ ] Expose retry telemetry through the download API.
- [ ] Render retry state in the Signal Room download queue.

- [ ] Add backend regression tests for retry, backoff, cancellation, pause, and restart recovery.
- [ ] Rebuild and validate the Windows executable.
- [ ] Save a final checkpoint and deliver the implementation status.

## Retry telemetry completion record

- [x] ~~Durable Hub transfer retry and exponential backoff implemented with a maximum of five configured retries.~~
- [x] ~~Retry count, attempt count, delay, next retry time, last error, and retry history persisted in each download job.~~
- [x] ~~API accepts and returns retry budget and telemetry through the existing download queue endpoints.~~
- [x] ~~Signal Room queue displays retry progress, backoff delay, next retry time, and transient errors.~~
- [x] ~~Backend suite passed with 121 tests.~~
- [x] ~~Windows executable rebuilt and smoke-tested for UI, API, authentication, machine detection, and download queue availability.~~

## Current Task — Referenced GUI Implementation

- [ ] Read the referenced task conversation and extract the GUI requirements.
- [ ] Map requirements to the existing Signal Room screens and preserved workflows.
- [ ] Define the new GUI information architecture and visual direction.
- [ ] Implement the new GUI in the preview source.
- [ ] Verify responsive layouts and key interaction states.
- [ ] Rebuild packaged web assets and the Windows executable.
- [ ] Save a final checkpoint and deliver the updated GUI.

## Referenced GUI redesign completion record

The existing Signal Room was redesigned in place as a calm neutral AI productivity workspace. The implementation preserves routes, APIs, authentication, integrations, local-model activation, retry telemetry, and the existing Windows launcher. A contextual Preview

/ Files / Activity / Details rail was added, the task intake was made a white document-like composer, and responsive collapse behavior was added for smaller screens. Preview builds passed, desktop and mobile visual checks passed, the Windows executable was rebuilt, and packaged UI/API/provider/local-model smoke checks returned HTTP 200.

## Current Task — Attachments, Context Links, Activity, and Manus Connectors

- [ ] Inspect current attachment-capable workspace APIs and frontend task submission contracts.
- [ ] Inspect current run, artifact, file, and activity data contracts for deep links and timeline rendering.
- [ ] Read connector configuration guidance and inspect enabled Manus connectors.
- [ ] Implement real file attachment selection and safe submission through existing workspace APIs.
- [ ] Add contextual rail deep links for selected runs, files, and artifacts.
- [ ] Add compact live activity timeline updates during run streaming.
- [ ] Add connector-aware settings and connection status for available Manus connectors.
- [ ] Verify preview, APIs, responsive UI, and Windows executable packaging.
- [ ] Save a final checkpoint and deliver the expanded GUI.

## Expanded GUI completion record

Implemented browser file attachment selection with bounded readable-file context, removable attachment chips, searchable full Manus connector catalog with enabled-state gating, connector IDs forwarded in task environment, contextual deep links into runs/files/artifacts, and a compact live activity timeline backed by streamed run events. Preview production build passed; desktop and mobile screenshots passed; the full Windows backend suite passed with 121 tests; the standalone `Orville-Signal-Room.exe` was rebuilt and validated with UI 200, docs 200, unauthenticated API 401, authenticated health 200, local models 200, and Hub download queue 200.

## Current Task — Use the Manus Connector Catalog

- [ ] Record the supplied 372-connector catalog and distinguish enabled connectors from catalog-only entries.

- [ ] Define a safe standalone connector-bridge boundary; do not execute arbitrary connector commands or expose secrets.
- [ ] Add backend connector catalog, bridge configuration, health, invocation, approval, audit, and failure routes.
- [ ] Add per-run connector invocation controls to the Signal Room.
- [ ] Preserve connector IDs in objective context and keep authentication outside task state.
- [ ] Add backend regression tests using a local fake connector bridge.
- [ ] Rebuild and validate the Windows executable and connector execution smoke flow.
- [ ] Save a final checkpoint and deliver the connector usage status.

## Manus connector execution completion record

Implemented a safe HTTP connector bridge with bounded URL, UID, operation, argument, response, and timeout validation. Added authenticated catalog-status, health, and approved invocation API routes with redacted audit records and explicit failure responses. Added Signal Room connector operation, JSON arguments, bridge-health, and approve-and-invoke controls while retaining the full 372-entry catalog and enabled-state gating. Added standalone connector bridge documentation and a local fixture smoke server. Backend suite passed with 123 tests; preview build passed; the Windows executable was rebuilt and packaged connector health/invocation smoke testing passed.

## Current Task — Connector Operation Discovery Demo Video

- [ ] Define the product-demo sequence for selecting a connector and discovering operations.
- [ ] Generate a concise Signal Room demo video.
- [ ] Review the generated video for sequence clarity and deliver it.

## Current Task — Animated Connector Discovery Prototype

- [ ] Define prototype states for connector selection, operation discovery, permissions, request preview, and approval.
- [ ] Implement the animated HTML prototype in the existing Signal Room preview.
- [ ] Verify interactions, animation timing, accessibility, and mobile layout.

- [ ] Save a checkpoint and deliver the prototype.

## Current Task — Connector Discovery Storyboard Images

- [ ] Define a coherent set of frames for connector selection, operation discovery, permissions, request preview, and approval.
- [ ] Generate the storyboard images with consistent Signal Room styling.
- [ ] Review image readability and deliver the set.

## Connector discovery storyboard completion record

Generated a four-frame visual sequence showing connector selection, operation discovery, permission and request review, and explicit approval/invocation in the established warm-neutral Signal Room style. The images are available through the generated project assets and may be used as a product walkthrough or design reference.

## Current Task — Windows Release Hardening

- [ ] Inspect launcher, PyInstaller spec, storage paths, and available Windows packaging tools.
- [ ] Define user-data, portable-mode, migration, and recovery boundaries without changing the GUI.
- [ ] Add native application-window support using the existing web bundle.
- [ ] Add dynamic local-port selection and communicate selected ports to the unchanged GUI.
- [ ] Add single-instance protection and orphan-process cleanup.
- [ ] Add crash recovery diagnostics and repair-safe startup behavior.
- [ ] Add installer and portable ZIP build scripts.
- [ ] Add update-safe data migrations and release documentation.
- [ ] Add optional code-signing configuration without embedding credentials.
- [ ] Run backend, packaged, installer, portable, and recovery validation.
- [ ] Save a final checkpoint and deliver the hardened release artifacts.

## Current Workstream — Local Connector Bridge and Sign-In Menu

- [x] ~~Define connector connection/session data model and secret-handling boundary.~~
- [x] ~~Implement local bridge service with health, catalog, OAuth/device/manual sign-in, callback handling, and invocation routes.~~
- [x] ~~Add encrypted-at-rest or OS-protected credential storage with redacted status responses.~~
- [x] ~~Add per-connector allowlists, operation discovery, approval gates, timeouts, response limits, and audit records.~~
- [x] ~~Add dedicated Connectors menu to the Signal Room without changing the established visual design.~~
- [x] ~~Add connection wizard states for sign-in required, connected, expired, disabled, and error.~~
- [x] ~~Add tests for auth, callback, persistence, token redaction, approval, allowlists, timeout, and restart recovery.~~
- [x] ~~Package and run Windows end-to-end connector smoke tests.~~
- [x] ~~Update connector bridge and user setup documentation.~~

# 18. Manus-Parity Roadmap — Excluding Cloud Browser

**Roadmap purpose:** Implement every unavailable or incomplete capability identified in the Manus parity analysis while preserving Orville’s local-first Windows executable, Signal Room visual design, explicit approvals, and fail-closed security posture. Cloud Browser is excluded by design. Local browser access remains in scope through the existing Playwright adapter and a future Chrome/Edge Browser Operator extension.

**Research basis:** Official Manus API v2 and product documentation reviewed on 2026-08-26. Source index: `/home/ubuntu/orville_manus_parity_sources.md`. Detailed analysis: `/home/ubuntu/Orville_Manus_Parity_Gap_Analysis.md`.

## 18.1 Roadmap rules and non-goals

- [ ] Preserve the existing Signal Room visual system and native Windows shell; new capabilities must extend existing surfaces rather than replace the GUI.
- [ ] Preserve local-first operation. Every cloud-dependent feature must have a documented local mode, an explicit optional hosted mode, or a clear unsupported state.
- [ ] Exclude Cloud Browser infrastructure from this roadmap. Implement only local browser access, local extension relay, takeover, allowlists, approvals, audit, and recovery.
- [ ] Never represent a catalogued connector as operational until its authentication, capability discovery, operation schema, and invocation tests pass.
- [ ] Treat credentials, OAuth tokens, cookies, files, prompts, browser content, connector responses, and generated code as separate trust domains.
- [ ] Require independent verification for every material feature, including security tests, restart tests, failure-path tests, and user-visible acceptance tests.
- [ ] Keep all new services runnable outside Manus through documented Python/Node commands, configuration files, migration steps, and test fixtures.
- [ ] Document cost boundaries before enabling any hosted provider, external API, persistent relay, notification channel, or paid model.

## 18.2 Phase A — Durable task-thread protocol [P0]

### A1. Task and message model

- [ ] Add a durable `TaskThread` model with stable task ID, project ID, agent ID, parent task ID, status, stop reason, active model, connector set, skill set, structured-output state, timestamps, and recovery metadata.
- [ ] Add an append-only `TaskMessage` model for user messages, assistant messages, tool calls, tool results, status updates, questions, approvals, errors, artifacts, and citations.
- [ ] Add explicit statuses: `planned`, `ready`, `running`, `waiting`, `stopped`, `failed`, `cancel_requested`, `cancelled`, and `recovering`.
- [ ] Add explicit stop reasons: `finish`, `ask`, `approval_required`, `cancelled`, `error`, `timeout`, and `policy_blocked`.
- [ ] Implement `send_message`, `list_messages`, `task_detail`, `stop`, `resume`, and `retry` operations.

- [ ] Preserve full event history across process restart and migration.
- [ ] Add optimistic concurrency/version numbers so duplicate user actions cannot advance a task twice.

### A2. Waiting and confirmation protocol

- [ ] Define a typed `WaitingRequest` with event ID, event type, description, JSON Schema, risk classification, requested permissions, expiry, and originating tool.
- [ ] Implement `ask_user` for normal questions and `confirm_action` for every other approval-gated event.
- [ ] Validate confirmation payloads against the stored JSON Schema before execution.
- [ ] Add confirmation types for terminal execution, file writes, repository changes, browser takeover, form submission, download, connector invocation, account changes, payments, deployment, secret entry, and model installation.
- [ ] Add “allow once”, “allow for task”, and “always allow for this safe scope” policies with explicit expiry and revocation.
- [ ] Prevent a rejected or expired confirmation from silently advancing the task.
- [ ] Add UI rendering for schema-driven confirmation forms with safe defaults and irreversible-action warnings.

### A3. Acceptance criteria

- [ ] A task can receive at least three follow-up messages without losing context.
- [ ] A task paused for a question remains paused until a user response is received.
- [ ] A task paused for an approval resumes only after a valid schema-conforming approval.
- [ ] Restarting the executable during `running` or `waiting` restores the correct state without duplicate tool execution.
- [ ] Every state transition is visible in the activity timeline and persisted in the audit log.

## 18.3 Phase B — Agent registry and subtask runtime [P0]

- [ ] Create an `AgentProfile` model with stable ID, name, description, system instructions, model policy, memory scope, skill set, connector set, tool permissions, risk ceiling, and enabled state.

- [ ] Convert the existing Personal Agent into a real registry-backed agent with a persistent main thread.
- [ ] Add agent creation, update, clone, disable, delete, and inspect operations.
- [ ] Add child-task creation with parent/child relationships, bounded depth, budgets, deadlines, and cancellation propagation.
- [ ] Add subtask result contracts containing status, artifacts, citations, errors, metrics, and verification record.
- [ ] Add parallel subtask execution with queue limits and explicit owned-path/resource claims.
- [ ] Add synthesis stages that cannot complete until required child tasks meet their verification policy.
- [ ] Add failure policies: retry child, skip optional child, pause for user, or fail parent.
- [ ] Add per-agent tool and connector permission policies.
- [ ] Add tests for nested subtasks, cancellation, timeouts, retries, partial completion, and restart recovery.

**Acceptance:** An agent can create three independent child tasks, execute them within bounded concurrency, collect their outputs, invoke a verifier, and publish one parent result with traceable lineage.

## 18.4 Phase C — Skills system [P0]

- [ ] Define a skill package format containing metadata, `SKILL.md`, version, author, license, permissions, dependencies, entry points, and optional resources.
- [ ] Implement local folder, ZIP, official package, and GitHub repository import.
- [ ] Validate package paths, archive traversal, symlinks, executable content, dependency declarations, and unsafe commands.
- [ ] Add static inspection and risk report before installation or first execution.
- [ ] Add skill registry with installed, disabled, update-available, incompatible, and quarantined states.
- [ ] Implement version pinning, update checks, rollback, and uninstall.
- [ ] Implement progressive disclosure: metadata at startup, instructions on activation, resources on demand.

- [ ] Add slash-command skill activation and task-level skill selection.
- [ ] Add automatic skill recommendation only after user approval or explicit project policy.
- [ ] Run skills inside the same sandbox, approval, timeout, network, and audit boundary as tools.
- [ ] Add a skill authoring wizard that converts a verified workflow into a package.
- [ ] Add malicious-skill fixtures and regression tests.

**Acceptance:** A user can import a local skill, inspect its permissions, approve it, invoke it from the composer, observe its tool calls, disable it, and verify that disabled skills cannot execute.

## 18.5 Phase D — Connector adapter platform [P0]

### D1. Adapter contract

- [ ] Define a versioned `ConnectorAdapter` interface for metadata, authentication, refresh, revoke, health, capability discovery, operation schemas, invocation, pagination, uploads, downloads, error normalization, and rate limits.
- [ ] Add connector states: `catalogued`, `supported`, `authorization_required`, `connected`, `expired`, `reauthorization_required`, `disabled`, `rate_limited`, `degraded`, and `unsupported`.
- [ ] Store connector manifests separately from credentials.
- [ ] Add official provider URLs, scopes, redirect URIs, API versions, and documentation references to each supported manifest.
- [ ] Add per-operation risk class and approval policy.
- [ ] Add request/response schema validation and redaction rules.

### D2. Initial provider set

- [ ] Implement and test Gmail.
- [ ] Implement and test Google Calendar.
- [ ] Implement and test Slack.
- [ ] Implement and test Notion.
- [ ] Implement and test GitHub.
- [ ] Implement and test Microsoft Outlook Mail.

- [ ] Implement and test Stripe in read-only mode first, then approved write actions.
- [ ] Implement and test HubSpot or another CRM provider.
- [ ] Implement and test Zapier or n8n as an automation provider.
- [ ] Add a generic OpenAPI/HTTP adapter for user-owned services with explicit allowlists.
- [ ] Maintain the remaining catalog entries as catalogued or unsupported until real adapters exist.

### D3. Auth and operations

- [ ] Support provider-specific OAuth2 authorization-code + PKCE.
- [ ] Support API-key, bearer, signed-request, and local endpoint authentication where appropriate.
- [ ] Add token refresh, expiry detection, revocation, reauthorization, and account labeling.
- [ ] Add per-provider redirect/callback tests using local fixtures.
- [ ] Add provider-specific pagination, retry, rate-limit, and error handling.
- [ ] Add connector defaults at user, project, and task levels.
- [ ] Implement explicit connector override, clear, and reuse semantics for follow-up task messages.
- [ ] Add operation discovery and schema-driven invocation UI.
- [ ] Add audit records that never store raw credentials or authorization headers.
- [ ] Add connector health checks and “test connection” actions that do not perform mutations.

**Acceptance:** At least eight high-value providers complete a full sign-in → refresh → discovery → read operation → approval-gated write operation → revoke flow using real or provider-approved test environments.

## 18.6 Phase E — Durable scheduler [P0]

- [ ] Define schedule model with task template, project, agent, connector set, skill set, timezone, recurrence, next run, state, retry policy, concurrency policy, and missed-run policy.
- [ ] Support one-time, interval, daily, weekday, weekly, monthly, and cron schedules.
- [ ] Add pause, resume, edit, clone, run-now, and delete actions.

- [ ] Persist schedules independently of the GUI process.
- [ ] Add worker leasing so only one process executes a scheduled run.
- [ ] Add catch-up, skip, and coalesce policies for missed runs.
- [ ] Add execution history with outputs, artifacts, errors, costs, connector actions, and approvals.
- [ ] Add notifications for success, failure, waiting, approval, connector expiry, and repeated retries.
- [ ] Add schedule import/export and backup coverage.
- [ ] Test daylight-saving changes, clock skew, restart, sleep/wake, duplicate execution, and long-running tasks.

**Acceptance:** A recurring task runs correctly while the GUI is closed, survives restart, produces one execution record per scheduled run, and never duplicates a run after process recovery.

## 18.7 Phase F — Webhooks and event delivery [P0]

- [ ] Add webhook endpoint registration, update, disable, rotate-secret, test, and delete operations.
- [ ] Support local loopback callbacks and documented secure relay configuration.
- [ ] Validate webhook payloads and enforce maximum body size.
- [ ] Add HMAC or asymmetric signature verification and timestamp/replay protection.
- [ ] Add idempotency keys and a deduplication store.
- [ ] Add exponential backoff with jitter, retry caps, dead-letter state, and manual replay.
- [ ] Add delivery history with status, latency, response code, retry count, and redacted error.
- [ ] Add task-created, task-status-changed, task-waiting, task-stopped, artifact-created, connector-expired, and schedule-failed events.
- [ ] Add webhook policy controls so external events cannot bypass approval gates.
- [ ] Add fixture tests for duplicate, delayed, malformed, unsigned, and replayed events.

**Acceptance:** A signed callback can resume a waiting task exactly once, failed deliveries retry safely, and unsigned or replayed payloads are rejected and audited.

## 18.8 Phase G — Structured output [P0]

- [ ] Add JSON Schema input to task creation and follow-up messages.
- [ ] Implement supported-subset validation before execution.
- [ ] Enforce object root, `additionalProperties: false`, required fields, nesting depth, and supported types/keywords.
- [ ] Persist schema state as `armed`, `paused`, `consumed`, `failed`, or `rearmed`.
- [ ] Extract only after a successful terminal completion.
- [ ] Preserve schema when a task pauses for user input.
- [ ] Return `{success, value, error}` with a schema-conforming zero-value fallback on extraction failure.
- [ ] Display structured results as JSON, table, downloadable artifact, and task event.
- [ ] Add schema fixtures for research extraction, code manifests, connector results, and data analysis.

**Acceptance:** Valid schemas produce deterministic structured artifacts; invalid schemas fail before the model run; asking a question does not consume the schema.

## 18.9 Phase H — Files and project knowledge bases [P1]

- [ ] Add managed file records with ID, filename, MIME type, size, hash, status, owner, project, task, created time, expiry, and deletion state.
- [ ] Implement safe local upload staging and optional S3-compatible storage abstraction.
- [ ] Add file type, size, archive, executable, and script policies.
- [ ] Add resumable uploads, checksum verification, cleanup, and quota reporting.
- [ ] Add file previews for text, images, PDFs, CSV, JSON, and code.
- [ ] Add project knowledge-base ingestion, chunking, indexing, retrieval, citations, and deletion propagation.
- [ ] Add project-scoped permissions and inherited instruction/version semantics.
- [ ] Add project pinning, ordering, filters, favorite tasks, and task-to-project movement.
- [ ] Add local-only and hosted-storage modes with explicit data-location indicators.
- [ ] Add backup/restore coverage for metadata and content.

**Acceptance:** A file can be uploaded, indexed into a project, cited by a task, deleted, and verified absent from retrieval results after cleanup.

## 18.10 Phase I — Local Browser Operator extension [P1; Cloud Browser excluded]

- [ ] Define a Chrome/Edge extension protocol for authorized tabs, sessions, screenshots, navigation, DOM extraction, and action requests.
- [ ] Implement authenticated local relay bound to loopback with origin validation and rotating session keys.
- [ ] Require explicit per-session browser authorization.
- [ ] Preserve “no password storage” and allow takeover for passwords, MFA, CAPTCHA, and sensitive pages.
- [ ] Add tab selection and browser-profile selection.
- [ ] Enforce domain allowlists and operation-level approvals.
- [ ] Add download path containment and file-type policies.
- [ ] Add visible stop/release control that immediately ends agent control.
- [ ] Add action timeline, screenshots, URL history, and redacted audit events.
- [ ] Add extension disconnect and browser restart recovery.
- [ ] Test Chrome and Edge, multiple tabs, stale sessions, MFA handoff, downloads, form submission, and emergency stop.

**Acceptance:** Orville can request control of an authorized local tab, complete an approved workflow, pause for takeover, return control to the user, and prove through audit records that credentials were not stored.

## 18.11 Phase J — Wide Research [P1]

- [ ] Add explicit Wide Research task mode with item source, item identity, requested fields, output format, concurrency, and evidence policy.
- [ ] Implement bounded map workers with isolated context per item.
- [ ] Add work queue, leases, progress counters, retries, partial completion, and resume.
- [ ] Store per-item source URLs, quotes, extracted values, uncertainty, and verification status.
- [ ] Add synthesis worker that waits for required items or produces a clearly marked partial result.

- [ ] Add table, CSV, JSON, Markdown report, and chart artifacts.
- [ ] Add rate-limit-aware concurrency and provider budget controls.
- [ ] Add cancellation that stops new items and allows active items to finish or terminate safely.
- [ ] Add tests for 10, 50, and 100-item fixtures with injected failures and duplicate sources.

**Acceptance:** Every item receives an independent context, failures are isolated, progress is visible, citations remain attached to rows, and synthesis identifies incomplete or uncertain items.

## 18.12 Phase K — Website build, publish, and artifact lifecycle [P1]

- [ ] Add website entity linked to project/task with title, visibility, URL, status, and current checkpoint.
- [ ] Add checkpoint records with version ID, commit/hash, status, timestamp, message, files, tests, and preview URL.
- [ ] Add local preview and optional user-selected hosting adapter.
- [ ] Add publish, republish-latest, update metadata, visibility, and rollback controls.
- [ ] Add deployment logs, health checks, failure state, and retry.
- [ ] Add custom-domain configuration documentation without storing provider secrets in project files.
- [ ] Add explicit deployment approvals and public-visibility confirmation.
- [ ] Add website artifact links from task history and contextual rail.

**Acceptance:** A website task produces a versioned checkpoint, preview, publish record, visible deployment state, and recoverable prior version.

## 18.13 Phase L — Slides and multimedia [P1]

### L1. Slides

- [ ] Add presentation artifact model with deck metadata, slide objects, notes, theme, assets, and source citations.
- [ ] Add outline → content → visual → review → export pipeline.

- [ ] Support editable PPTX, PDF, web slides, and speaker-notes export.
- [ ] Support imported templates with font, color, layout, and asset validation.
- [ ] Add chart generation from CSV/XLSX/JSON data.
- [ ] Add slide-level approval, revision, and visual verification.
- [ ] Add tests for deck generation, export, notes, and template preservation.

### L2. Multimedia

- [ ] Add unified media artifact model for image, audio, video, transcript, caption, and derived metadata.
- [ ] Add image understanding and OCR task stages.
- [ ] Add video ingestion, frame extraction, audio extraction, transcript alignment, and evidence timestamps.
- [ ] Add speech-to-text provider/local runtime integration.
- [ ] Add text-to-speech provider/local runtime integration.
- [ ] Add media generation job polling, cancellation, retries, quota, and asset cleanup.
- [ ] Add content-type capability negotiation before provider calls.
- [ ] Add approval and policy checks for generated media and external publishing.

**Acceptance:** A task can ingest a media file, produce derived artifacts with timestamps/citations, and expose all outputs in task history and the contextual rail.

## 18.14 Phase M — Usage, budgets, and provider health [P1]

- [ ] Record model calls, tokens, latency, retries, local compute time, connector calls, downloads, storage, and bandwidth.
- [ ] Add provider and connector rate-limit state with reset timestamps.
- [ ] Add per-task, per-project, per-agent, and global budgets.
- [ ] Add warning thresholds, hard stops, and approval escalation.
- [ ] Add usage dashboard with filters, pagination, export, and redacted diagnostics.
- [ ] Add health dashboard for models, runtimes, connectors, browser relay, scheduler, and webhooks.
- [ ] Add exponential backoff with jitter and circuit breakers for external services.

- [ ] Add cost-estimation disclaimers and provider-specific pricing configuration without hardcoding unstable prices.

**Acceptance:** A task stops or requests approval when its budget is exhausted, provider failures are visible, and retry behavior is measurable.

## 18.15 Phase N — Collaboration and remote local-host access [P2]

- [ ] Define hosted collaboration mode separately from local-only mode.
- [ ] Add identity, invitations, roles, permissions, task sharing, project sharing, and revocation.
- [ ] Add real-time event synchronization with ordered prompts and conflict handling.
- [ ] Add owner-controlled approval and connector permissions.
- [ ] Add secure remote gateway for submitting tasks to an online Windows host.
- [ ] Add device registration, revocation, session expiry, and notification controls.
- [ ] Add privacy indicators showing which files, connectors, and browser sessions are shared.
- [ ] Add collaboration audit history and export.

**Acceptance:** Two authorized users can view and prompt one task, see ordered updates, respect owner permissions, and revoke access without exposing unrelated projects or credentials.

## 18.16 Phase O — Security, recovery, and release operations [P0/P1]

- [ ] Add per-task working-directory isolation and safe path containment.
- [ ] Add subprocess CPU, RAM, wall-clock, output-size, and process-count limits.
- [ ] Add network egress policy by provider, connector, domain, and task.
- [ ] Add executable/script scanning and quarantine before execution.
- [ ] Add tamper-evident audit export with redaction verification.
- [ ] Add encrypted backup and restore for SQLite, catalogs, projects, task history, and protected connection metadata.

- [ ] Add migration dry-run, rollback-safe migration, and restore validation.
- [ ] Add signed update manifest, integrity validation, release channels, and rollback package.
- [ ] Add opt-in crash reporting with local redaction preview.
- [ ] Add clean-machine tests for first launch, missing runtime, blocked port, firewall, WebView2 absence, no network, corrupted state, interrupted migration, and duplicate launch.
- [ ] Add release SBOM, dependency audit, license inventory, and reproducible build record.

## 18.17 Verification matrix

- [ ] Unit-test every new model, migration, schema, policy, and state transition.
- [ ] Add fixture servers for OAuth, connector APIs, webhook delivery, file storage, provider rate limits, and browser relay.
- [ ] Add property tests for path containment, credential redaction, idempotency, retry bounds, and schema validation.
- [ ] Add integration tests for task → subtask → connector → approval → artifact → webhook flows.
- [ ] Add restart tests during every durable state transition.
- [ ] Add Windows executable smoke tests after each P0 phase.
- [ ] Add desktop and mobile Signal Room visual checks without changing the established design language.
- [ ] Add accessibility checks for keyboard navigation, focus order, labels, contrast, and reduced motion.
- [ ] Add performance tests for 100 concurrent local subtasks, large repositories, large files, and long event streams.
- [ ] Add a second-agent verification record for each completed roadmap phase.

## 18.18 Recommended execution order

- [ ] Release 1: Durable task threads, typed waiting/confirmations, structured output, and restart recovery.

- [ ] Release 2: Agent registry, bounded subtasks, skills registry, and sandbox permissions.
- [ ] Release 3: Connector adapter framework and first eight provider adapters.
- [ ] Release 4: Durable scheduler, webhooks, usage budgets, and provider health.
- [ ] Release 5: Managed files, project knowledge bases, and local Browser Operator extension.
- [ ] Release 6: Wide Research, website lifecycle, Slides, and unified multimedia artifacts.
- [ ] Release 7: Collaboration, secure remote access, backup/restore, signed updates, and operations hardening.

## 18.19 Definition of parity for this roadmap

- [ ] Orville can run a persistent multi-turn task with typed questions and approvals.
- [ ] Agents can create bounded child tasks and synthesize verified results.
- [ ] Skills can be imported, audited, permissioned, activated, disabled, and rolled back.
- [ ] Supported connectors can be signed into, refreshed, discovered, invoked, revoked, and audited.
- [ ] Scheduled tasks and webhooks operate safely while the GUI is closed.
- [ ] Structured output, files, projects, and citations have durable lifecycle semantics.
- [ ] Local browser access supports extension-based control and user takeover without storing passwords.
- [ ] Wide Research supports bounded parallel work with item-level evidence.
- [ ] Websites, presentations, media, and other artifacts have versioned preview/export/publish workflows.
- [ ] Usage, budgets, provider health, backups, updates, and recovery are visible and testable.
- [ ] Collaboration and remote access are either implemented with a secure hosted layer or explicitly marked unavailable in local-only mode.

## 18.20 Research references

- [ ] Read and implement against Manus API v2 task lifecycle: https://open.manus.im/docs/v2/task-lifecycle

- [ ] Read and implement against Manus API v2 agents: https://open.manus.im/docs/v2/agents-overview
- [ ] Read and implement against Manus API v2 connectors: https://open.manus.im/docs/v2/connectors
- [ ] Read and implement against Manus API v2 structured output: https://open.manus.im/docs/v2/structured-output
- [ ] Read and implement against Manus API v2 webhooks: https://open.manus.im/docs/v2/webhooks-overview
- [ ] Read and implement against Manus API v2 files: https://open.manus.im/docs/v2/file.upload
- [ ] Read and implement against Manus API v2 websites: https://open.manus.im/docs/v2/website
- [ ] Read and implement against Manus API v2 rate limits: https://open.manus.im/docs/v2/rate-limits
- [ ] Review Manus Skills: https://manus.im/docs/features/skills
- [ ] Review Manus Projects: https://manus.im/docs/features/projects
- [ ] Review Manus Desktop/My Computer: https://manus.im/docs/features/desktop
- [ ] Review Manus Browser Operator: https://manus.im/docs/features/browser-operator
- [ ] Review Manus Wide Research: https://manus.im/docs/features/wide-research
- [ ] Review Manus Scheduled Tasks: https://manus.im/docs/features/scheduled-tasks
- [ ] Review Manus Collab: https://manus.im/docs/features/collab
- [ ] Review Manus Slides: https://manus.im/docs/features/slides
- [ ] Review Manus Multimedia: https://manus.im/docs/features/multi-modal

## 18.21 Research limitation

- [ ] Video demonstrations were identified for additional first-hand evidence, but automated video analysis was unavailable during this run because the analysis service reported insufficient credits. Official documentation was used as the authoritative implementation basis; video evidence should be added during a later research pass if available.

# 19. Active Execution Batch — Manus-Parity Implementation

- [ ] Audit the current Windows repository, test baseline, packaging configuration, and existing roadmap state.
- [x] ~~Complete durable task-thread and message persistence with restart recovery.~~
- [x] ~~Complete schema-driven waiting and confirmation events.~~
- [x] ~~Complete structured-output validation, extraction, and artifact delivery.~~
- [ ] Complete agent registry and bounded subtask execution.
- [x] ~~Complete reusable Skills import, audit, permissions, activation, and rollback.~~
- [ ] Complete provider-specific connector adapter framework and priority adapters.
- [ ] Complete connector defaults, refresh, revoke, discovery, rate limits, and operation schemas.
- [ ] Complete durable scheduling and signed webhook delivery.
- [ ] Complete usage, budget, quota, provider-health, and notification controls.
- [ ] Complete managed file lifecycle and project knowledge-base indexing.
- [ ] Complete local Browser Operator extension and secure relay; Cloud Browser remains excluded.
- [ ] Complete Wide Research map/reduce execution and evidence synthesis.
- [ ] Complete website lifecycle, publishing adapter, and checkpoint management.
- [ ] Complete Slides artifact generation and export.
- [ ] Complete unified multimedia artifacts, speech-to-text, text-to-speech, and video understanding.
- [ ] Complete collaboration and secure remote-local access boundaries where feasible without claiming hosted parity.
- [ ] Complete sandboxing, backups, restore tests, signed updates, observability, and release hardening.
- [ ] Run full regression, clean-machine startup, security, recovery, and Windows packaging validation.

- [ ] Create final checkpoint and deliver the completed artifacts with documented limitations.

## 19.1 Execution progress — 2026-08-26

- [x] ~~Durable task-thread store, append-only messages, state transitions, restart recovery, typed waits, approvals, and structured outputs implemented and API-exposed.~~
- [x] ~~Persistent agent profiles, bounded child-task lineage, depth/child limits, disabled-agent enforcement, and cancellation propagation implemented and API-exposed.~~
- [x] ~~Local Skills registry implemented with folder/ZIP import, explicit approval, permission checks, archive traversal rejection, checksum metadata, enable/disable, quarantine, uninstall, and API routes.~~
- [x] ~~Scheduler execution history and persistent webhook event idempotency implemented; timestamped HMAC replay protection and payload limits added.~~
- [x] ~~Usage metering, local budgets, provider-health state, circuit opening, cooldown, and API endpoints implemented.~~
- [x] ~~Provider-neutral connector adapter contract, risk-classified operation manifests, generic HTTP adapter, response redaction, and priority manifests implemented.~~
- [ ] Provider-specific connector network handlers, OAuth refresh/revocation for each provider, and production credential test coverage remain incomplete.
- [ ] Signal Room UI still needs dedicated surfaces for agent profiles, Skills, task-thread state, budgets, provider health, and adapter support status.

## 19.2 Execution progress — continuation

- [x] ~~Provider-neutral connector adapter registry implemented with operation schemas, risk classes, safe generic HTTP policy, redacted results, and eight priority manifests.~~
- [x] ~~Local Browser Operator relay implemented with paired sessions, domain validation, action allowlists, explicit takeover approval, queue polling, expiry, and revocation.~~
- [x] ~~Connector-adapter, browser-relay, scheduler, webhook, usage, agent, Skills, task-thread, and Wide Research regression coverage passes.~~
- [x] ~~Bounded Wide Research runner implemented with isolated item execution, retries, cancellation, evidence fields, durable state, and resume behavior.~~

- [ ] Provider-specific network handlers and real credentialed integration tests remain required before claiming operational support for each connector.
- [ ] GUI surfaces and packaged-release rebuild remain required for this execution batch.

# 20. Full Connector Adapter Execution

- [ ] Inventory and normalize every connector catalog entry.
- [ ] Classify each connector as native-provider, generic-HTTP, OpenAPI-discoverable, local-endpoint, or configuration-required.
- [ ] Add a universal adapter manifest with authentication, base URL, scopes, operations, schemas, risk classes, limits, and documentation links.
- [ ] Add OpenAPI discovery with schema sanitization, operation caps, host allowlists, and user approval.
- [ ] Add provider-specific auth refresh, revoke, pagination, retry, and error normalization contracts.
- [ ] Generate a manifest for every catalog connector without falsely marking unsupported services operational.
- [ ] Connect manifests to per-connector sign-in, defaults, operation discovery, approval, audit, usage, and provider-health state.
- [ ] Add adapter support-state visibility to the Connectors menu.
- [ ] Add fixture and contract tests for every adapter class and priority provider group.
- [ ] Rebuild and validate the Windows executable and portable release.
- [ ] Document configuration-required connectors and the user steps for provider OAuth/API registration.

## 20.1 Full-catalog adapter milestone — 2026-08-26

- [x] ~~Inventory verified: 372 total connector entries, 5 catalog-enabled, 367 catalog-disabled.~~
- [x] ~~All 372 entries now have packaged adapter records with identity, description, catalog state, support state, supported auth modes, and transparent configuration-required status.~~
- [x] ~~Authenticated catalog search endpoint implemented at `/api/v1/connector-adapter-catalog`.~~

- [x] ~~Connectors UI now reads adapter support state and displays CONFIGURE, READY, or CONNECTED instead of implying every catalog item is operational.~~
- [x] ~~Windows executable rebuilt with the complete connector catalog embedded; portable release regenerated with extension bundle and documentation.~~
- [x] ~~Final executable validation passed: API health 200, adapter catalog returned 372 records, connector connections 200, agents 200, Skills 200, usage 200, browser relay 200, UI 200, extension manifest present.~~
- [ ] Provider-specific handlers, OAuth presets, real provider operation contracts, and credentialed integration tests remain required for each external service.

# 21. Four-Layer Connector Architecture Execution

- [ ] Audit catalog/manifest registry coverage, authentication lifecycle, operation adapters, and approval/audit gateway.
- [ ] Add manifest versioning, capability metadata, scopes, limits, risk classifications, documentation, and support-state transitions.
- [ ] Add OAuth2 PKCE refresh, revocation, expiry recovery, and provider preset lifecycle.
- [ ] Add API-key/bearer validation, rotation, account labels, and connection health checks.
- [ ] Add generic OpenAPI operation discovery with sanitization and approval.
- [ ] Add adapter pagination, upload/download contracts, bounded retries, rate-limit handling, and normalized errors/results.
- [ ] Integrate operation schemas with approval, redaction, egress policy, usage, and audit records.
- [ ] Add Connectors UI support-state, health, defaults, operation, and approval controls.
- [ ] Add fixture and contract tests for all four layers and rebuild the Windows release.

## 21.1 Four-layer architecture progress

- [x] ~~Manifest records now include version, capabilities, scopes, limits, operation output schemas, pagination metadata, and explicit support state.~~

- [x] ~~Catalog fallback manifests are accepted as configuration-required without falsely claiming provider support.~~
- [x] ~~OAuth connections now support protected refresh tokens, refresh-token rotation, optional provider revocation URLs, refresh recovery, and local disconnect.~~
- [x] ~~Operation discovery now uses the configured credential header and supports bounded JSON OpenAPI discovery with egress policy and risk classification.~~
- [x] ~~Connector documentation now describes all four layers, lifecycle boundaries, and the OpenAPI workflow.~~
- [ ] Connectors UI still needs dedicated controls for refresh, revoke, defaults, and OpenAPI discovery results.
- [ ] Provider-specific handlers and credentialed contract tests remain required for each external service.

## 21.2 Verified four-layer connector milestone

- [x] ~~Catalog and manifest registry exposes version, capabilities, scopes, limits, operation schemas, and support state.~~
- [x] ~~Authentication service supports DPAPI-protected manual credentials, OAuth2 PKCE, protected refresh tokens, refresh, optional provider revocation, expiry recovery, and local disconnect.~~
- [x] ~~Operation adapter service supports priority manifests, generic HTTP policy, credential-header-aware discovery, JSON OpenAPI discovery, pagination metadata, risk classification, timeouts, response limits, and redaction.~~
- [x] ~~Approval and audit gateway remains enforced for sensitive/critical actions and records connector lifecycle, discovery, invocation, failures, and policy blocks.~~
- [x] ~~Connectors UI exposes sign-in, refresh, provider revoke, disconnect, OpenAPI discovery, operation counts, and support-state indicators.~~
- [x] ~~Final packaged executable validation passed: health 200, 372 catalog records, 381 registry records, 9 operational priority manifests, protected connections 200, UI 200, connector route markers present in the packaged JavaScript, Browser Operator extension present, portable ZIP present.~~

- [ ] Provider-specific handlers and credentialed contract tests are still required for each third-party service; configuration-required fallback is intentional for services without a verified handler.

# 22. Connector Roadmap Continuation

- [ ] Implement user/project/task connector defaults with explicit override and clear semantics.
- [ ] Add provider OAuth presets, scopes, token endpoints, revocation endpoints, and refresh policy metadata.
- [ ] Add connector health, expiry, refresh, and reauthorization status controls to the Connectors UI.
- [ ] Add bounded adapter pagination and normalized page/cursor results.
- [ ] Add safe connector upload/download operation contracts with containment and size limits.
- [ ] Add adapter retries, rate-limit parsing, circuit integration, and normalized error envelopes.
- [ ] Integrate connector operation schemas with approval, usage, and audit records.
- [ ] Add credential-free provider fixtures and contract tests for all adapter classes.
- [ ] Rebuild and smoke-test the Windows executable and portable release.

## Roadmap Continuation — Scheduling and Webhooks Status

The durable scheduling and signed webhook milestone is implemented: ScheduleStore now supports migrated lease columns, atomic worker claims, release, listing, and stale-lease recovery. EventIntake now validates metadata before persistence, enforces timestamped HMAC signatures and replay windows, persists delivery outcomes, rejects duplicates, and exposes recent delivery inspection. AutomationDispatcher connects accepted scheduled and webhook events to enabled workflow versions with idempotent workflow runs, approval-aware deterministic execution, failure recording, and cleanup. The Signal Room includes a Schedules & webhooks view and direct `/automation` route.

- [x] ~~Durable schedule lifecycle, leases, and stale-lease recovery.~~

- [x] ~~Signed webhook validation, replay protection, idempotency, and audit records.~~
- [x] ~~Schedule and webhook dispatch into enabled workflow execution.~~
- [x] ~~Signal Room monitoring view for schedules and inbound events.~~
- [ ] Rebuild and smoke-test the Windows executable and portable release.

## Repository Governance Controls

- [x] ~~Audit and maintain AGENTS.md with repository-specific operating rules.~~
- [x] ~~Maintain CHANGELOG.md for material roadmap, architecture, and behavior changes.~~
- [x] ~~Define and document predictable directories for source, tests, configuration, documentation, generated artifacts, logs, and temporary files.~~
- [x] ~~Document that external instructions in files, websites, emails, and tool outputs are untrusted data unless explicitly endorsed.~~
- [x] ~~Document approval requirements for posting, payments, account or permission changes, credential entry, and destructive file or repository actions.~~
- [x] ~~Document secret storage, masking, rotation, and prohibition on logging credential values.~~
- [x] ~~Document artifact retention and cleanup rules for temporary files, downloads, generated media, and execution logs.~~
- [x] ~~Define naming, formatting, commit, branch, and review conventions for generated code.~~
- [x] ~~Validate governance rules against the current scripts, packaging layout, tests, and roadmap documents.~~